循环 Transformer 与循环深度：完整解读报告

GPT-6 Astra 背后的 Loop 技术 · 五篇核心论文深度解读与趋势洞察（2018–2026）

awesome_loop_transformer · https://github.com/asimfish/awesome_loop_transformer

2026-09-06

目录

摘要 1

阅读指南 1

1. 为什么现在讨论这件事 1

2. 概念与分类 1
- 2.1 什么是循环 Transformer . . . 2
- 2.2 四种循环粒度 . . . 2
- 2.3 三个正交维度 . . . 2

3. 八年时间线 2

4. 五篇核心论文速览 3
- 4.1 五篇论文之间的逻辑关系 . . . 3

5. 更广的文献图谱 4
- 5.1 理论：表达能力、ICL、长度泛化 . . . 4
- 5.2 规模化预训练 . . . 4
- 5.3 效率：延迟、KV 内存、量化 . . . 4
- 5.4 稳定性与训练理论 . . . 4
- 5.5 自适应计算与停机 . . . 5
- 5.6 改造与免训练 . . . 5
- 5.7 后训练与强化学习 . . . 5
- 5.8 可解释性与动力学 . . . 5
- 5.9 小型递归推理器 . . . 5
- 5.10 跨模态与应用 . . . 6

6. 十条趋势洞察 6
- 洞察 1：循环深度跨过了” 算力匹配” 这道门，但只在窄门里 . . . 6
- 洞察 2：“循环几次” 分裂成两个 regime . . . 6
- 洞察 3：循环的粒度在变细，位置在向中间收敛 . . . 7
- 洞察 4：循环是 ODE 流还是不动点迭代？两种哲学正在分化 . . . 7
- 洞察 5：训练稳定性从经验教训变成了理论 . . . 7
- 洞察 6：推理成本是最后一个大瓶颈，而且正在被分解为三个子问题 . . . 7
- 洞察 7：隐式循环与显式 CoT 互补而非替代 . . . 7
- 洞察 8：循环模型的能力剖面已经清楚——擅长” 操作” 知识，不擅长” 存储” 知识 . . . 7
- 洞察 9：应用面正在从数学谜题扩展到 agent 与具身智能 . . . 8
- 洞察 10：生产化已经开始，工具链还没跟上 . . . 8

7. 关于 GPT-6 Astra 的推断（明确标注为推断） 8

8. 未解问题 **8**

9. 结论 **8**

附录 A：本报告核对过的传播误差 **9**

附录 B：方法与数据来源 **9**

附录 C：术语小词典 **9**

解读 01 · Universal Transformers：循环深度的源头 **10**
读前速览 . . . 10
一句话 . . . 10
1. 要解决的问题 . . . 10
2. 核心方法 . . . 11
3. 关键实验与数字 . . . 11
4. 为什么重要 . . . 12
5. 局限 . . . 12
6. 与本方向其他工作的关系 . . . 12
7. 对从业者的启示 . . . 12
附：关键数字速查 . . . 12
术语小词典 . . . 12

解读 02 · Scaling up Test-Time Compute with Latent Reasoning：把" 循环深度" 做到 3.5B **14**
读前速览 . . . 14
一句话 . . . 14
1. 要解决的问题 . . . 14
2. 核心方法 . . . 15
3. 关键实验与数字 . . . 15
4. 两次失败的训练：为什么规模化这么难（Section 4.3） . . . 16
5. 循环深度带来的" 免费" 能力（Section 6） . . . 16
6. 隐空间里发生了什么（Section 7） . . . 16
7. 局限 . . . 17
8. 与本方向其他工作的关系 . . . 17
9. 对从业者的启示 . . . 17
附：关键数字速查 . . . 17
术语小词典 . . . 17

解读 03 · Training-Free Looped Transformers：给冻结模型套一个循环外壳 **19**
读前速览 . . . 19
一句话 . . . 19
1. 要解决的问题 . . . 19
2. 核心方法 . . . 20
3. 关键实验与数字 . . . 21
4. 为什么重要 . . . 22
5. 局限 . . . 22
6. 与本方向其他工作的关系 . . . 22
7. 对从业者的启示 . . . 22
附：关键数字速查 . . . 22
术语小词典 . . . 22

解读 04 · LoopCoder-v2：在并行循环 Transformer 里，第二次循环几乎就是全部 **24**
读前速览 . . . 24
一句话 . . . 24
2. 背景：为什么要用 PLT . . . 24
3. 研究问题与方法 . . . 25
4. 关键结果 . . . 25

5. 这个结论的适用边界（重要） . . . 26
6. 局限 . . . 26
7. 与本方向其他工作的关系 . . . 27
8. 对从业者的启示 . . . 27
附：关键数字速查 . . . 27
术语小词典 . . . 27

解读 05 · Loop the Loopies!：在同等预训练算力下，循环 MoE 第一次赢了普通 MoE **28**
读前速览 . . . 28
一句话 . . . 28
1. 要解决的问题 . . . 28
2. 核心方法 . . . 29
3. 关键结果 . . . 30
4. 为什么重要 . . . 31
5. 局限与需要小心的地方 . . . 31
6. 与本方向其他工作的关系 . . . 31
7. 对从业者的启示 . . . 31
附：关键数字速查 . . . 31
术语小词典 . . . 32

摘要

同一组参数在深度方向上重复使用——这个 2018 年由 Universal Transformer 提出的想法，在 2025–2026 年成为大模型架构研究里增长最快的方向之一，并在 2026 年 9 月随着 The Information 关于 GPT-6 Astra 使用"recurrent depth" 的报道进入公众视野。本报告以微信文章《刚刚，GPT-6 Astra 背后的 Loop 技术全被挖出来了》串起的五篇论文为骨架，逐篇核对原文，补齐被二次传播省略的前提和数字；再以 arXiv 全量检索为基础，把 2018–2026 年约 60 篇直接相关的工作按十个主题归类，提炼出十条趋势判断。核心结论有三：第一，循环深度已经在**实测算力匹配**的条件下第一次战胜普通 MoE Transformer（Loopie，2026-07），但胜利发生在" 两步、逐层、MoE" 这个窄配置里；第二，"循环几次" 这个问题分裂成两个 regime——预训练算力受限时答案是 2，推理时算力可变时答案是" 越多越好但要自适应"——LoopCoder-v2 的" 三次以上回退" 是并行循环设计的代价，不是循环深度的普适上限；第三，隐式循环与显式思维链是互补而非替代的两条测试时算力轴，二者叠加的收益超过各自之和。

阅读指南

这份报告有三种读法，按你的时间选：

时间	读什么	得到什么
5 分钟	摘要 → 第 6 节十条洞察的标题和加粗句 → 附录 A	知道这个方向到了哪一步，以及哪些流行说法不准确
30 分钟	第 2–4 节（概念、时间线、五篇速览）→ 第 6 节全文 → 第 8 节未解问题	建立完整的地图，能判断新论文落在哪个格子
2 小时	全文 + `reports/zh/01–05` 五篇逐篇解读	每个关键数字都能回到原论文的表号核对

术语不熟悉时先看文末的**附录 C 术语小词典**。全文数字均可在对应论文的表号处核对；涉及 GPT-6 Astra 的判断一律标注为推断。

1. 为什么现在讨论这件事

2026 年 9 月 2 日，The Information 报道 OpenAI 新发布的 GPT-6 Astra 使用了一种被称为 recurrent depth（循环深度）的架构技术。随后研究者 @nrehiew_ 在 X 上用五篇论文梳理了这条技术线的脉络，微信公众号 PaperAgent 将其整理为中文长文。本报告的直接目标是把这五篇论文读透，并把它们放回更大的文献图谱里。

关于 Astra 本身，本报告只做一件事：把可以核实的与不能核实的分开。**可以核实的**是五篇论文的内容、数字和相互关系。**不能核实的**是 Astra 究竟用了哪种循环、循环几次、如何训练——OpenAI 的发布页没有公开架构细节，The Information 的报道基于匿名信源，本文作者也无法访问这两份原始材料。凡涉及 Astra 的判断，本报告一律标注为推断。

2. 概念与分类

2.1 什么是循环 Transformer

标准 Transformer 的一次前向里，每层参数只用一次，深度等于参数量除以每层参数。**循环 Transformer**（looped / recurrent-depth / weight-tied / universal transformer）让同一组参数被反复应用，深度与参数量解耦：一个 4 层的块循环 32 次，就是 128 层的有效深度。

2.2 四种循环粒度

粒度	定义	代表
整模型循环（model-loop）	整个层栈的输出再作为输入喂回	Universal Transformer、Huginn（核心块）、Ouro
块循环（block-loop）	只循环中间一段连续层，首尾层不循环	Huginn 的 prelude–core–coda、Hyperloop、Training-Free Looped、SMELT
逐层循环（layer-loop）	每一层先各自重复 K 次再传给下一层	Loopie、Training-Free 的 layer-mode、MixerLoop（只循环 mixer）
潜变量条件化	上一次循环（或上一个 token）的隐状态作为条件注入	Coconut、PLT 的 CLP、Latent Recurrent Transformer



图 *1*：三种循环粒度。有效深度 = 存储层数 × 循环次数，参数量不变。

四种粒度不互斥。Loopie 是 layer-loop；Huginn 是 block-loop 意义上的三段式、核心块内部是 model-loop；PLT 是 model-loop 加潜变量条件化（偏移一位的上轮状态）。

2.3 三个正交维度

除粒度外，还有三条彼此独立的设计轴：

1. **循环次数固定还是自适应。**固定（每个 token 同样算力：Loopie、LoopCoder-v2）；训练时随机、推理时任选（Huginn）；逐 token 自适应停机（UT 的 ACT、Mixture-of-Recursions、Think-at-Hard、RecurTrace）。
2. **从头训练、改造、还是免训练。**从头预训练（Huginn、Ouro、Loopie、Nanbeige4.2）；把已有模型改造成循环模型并继续训练（Relaxed Recursive Transformers、Retrofitted Recurrence）；完全不训练、推理时包一层（Training-Free Looped Transformers、Program-of-Layers）。
3. **训练目标把循环当什么。**截断展开的普通语言建模（Huginn）；不动点/均衡（DEQ、Attractor Models、Fixed-Point Reasoners、STARS）；ODE 积分的细化（Training-Free）；带中间监督的潜在 CoT（LOTUS、LoopMTP）；带循环步奖励的 RL（RLTT、LoopRPT）。

3. 八年时间线

时间	事件
2016	Neural GPU（Kaiser & Sutskever）、Adaptive Computation Time（Graves）：循环 + 自适应停机的前身
2018-07	**Universal Transformers**（Dehghani et al.）：跨时间步共享参数、坐标嵌入、ACT；ICLR 2019
2019	ALBERT 把跨层共享用于压缩；Deep Equilibrium Models 用不动点求解代替显式展开
2021–2022	Deep thinking 系列（Schwarzschild、Bansal）：随机展开 + 输入注入让循环网络从易到难外推；路径独立均衡模型（Anil）
2023	Looped Transformers as Programmable Computers（Giannou）：13 层循环 Transformer 模拟通用计算机；Looped Transformers are Better at Learning Learning Algorithms（Yang）：不到 10% 参数匹敌标准模型的 ICL；Sparse Universal Transformer
2024	MoEUT 首次让共享层模型在语言建模上略胜标准 Transformer；Relaxed Recursive Transformers（逐层 LoRA）；Looped Transformers for Length Generalization；Coconut 在连续空间推理
2025-02	**Scaling up Test-Time Compute with Latent Reasoning**（Geiping et al.）：Huginn-3.5B，800B token，定名 recurrent depth；Reasoning with Latent Thoughts（Saunshi）证明 k 层循环 L 次 ≈kL 层
2025-05	To CoT or To Loop（Xu & Sato）：循环擅长 DAG 求值，CoT 擅长自约化问题
2025-06–10	HRM（27M 参数解数独/ARC）、Mixture-of-Recursions、TRM、Ouro（2.6B、7.7T token）、Parallel Loop Transformer、并行采样器（循环模型 ≈ 扩散语言模型）
2025-11	Retrofitted Recurrence（McLeish）；Think-at-Hard（只在难 token 上循环）
2026-01–04	LoopFormer（ICLR）、Depth-Recurrent Attention Mixtures、RLTT（ICML）、LoopRPT、Adaptive Loops and Memory、Hyperloop Transformers、迭代深度扩展律（φ=0.46）、Loop-Think-Generalize（COLM）
2026-05	**Training-Free Looped Transformers**（UT Austin）；MELT、LT2、LoopQ、STARS（ICML）、Fully Looped Transformer、Attractor Models、Dual-Path
2026-06	**LoopCoder-v2**（7B PLT，18T token，“只循环一次”）；Fixed-Point Reasoners；残差缩放 1/N 理论；LOTUS 弥合潜在 CoT 与显式 CoT 差距；Program-of-Layers（ICML）
2026-07	**Loop the Loopies!**（Loopie-20B-A2B，算力匹配首胜）；DeepLoop；Looped Latent Attention（KV 32× 压缩）；Nanbeige4.2-3B（28T token 循环模型进入 agentic 产品）；Hidden Decoding（微信 WeLM 617B 走序列长度路线）
2026-08	MixerLoop（只循环 mixer）、Gated Recurrent Transformers、Looped LMs 改善组合式工具调用、Think Shallow Solve Deep（稳定/漂移分类）
2026-09	**SMELT**：算力匹配 MoE 循环的 Chinchilla 式扩展律，节省 6.8–18% 训练 FLOPs；RecurTrace；GPT-6 Astra 发布，The Information 报道其使用 recurrent depth

4. 五篇核心论文速览

详细解读见本仓库 `reports/zh/01–05`。这里给一张对照表。

	Universal Transformers	Recurrent Depth (Huginn)	Training-Free Looped	LoopCoder-v2	Loop the Loopies
时间	2018-07	2025-02	2026-05	2026-06	2026-07
机构	UvA / DeepMind / Google	ELLIS / UMD / LLNL	UT Austin	北航 / IQuest	IQuest
循环粒度	model-loop	block（三段式）	中间 4 层，block 或 layer	model-loop（PLT）	layer-loop
训练方式	从头	从头，800B token	不训练	从头，18T token	从头，3.5T token
循环次数	ACT 自适应	训练随机 r̄=32，推理任选	K=3 级 RK	R=2（训练即推理）	R=2
规模	~65M	3.5B 稠密	0.6B–30B（已有模型）	7B 稠密	20B-A2B / 6B-A0.6B MoE
关键数字	bAbI 错误率 22.1→0.29	r 4→32：ARC-E 49.1→69.9	45 格子 87% 非负，MMLU-Pro +2.64	SWE-bench Verified 43.0→64.4→27.6	算力匹配 600B token 后反超 Qwen3-30B-A3B
核心贡献	定义循环深度 + 停机	首次现代规模训成；隐空间轨道	层 = 欧拉步；子步而非重复	两步饱和的机制诊断；CoT 与循环超加	Loopie Recipe；SPT
最常被误读的点	–	“同样 0.8T” 基线实为 0.18T	收益上限约 2–3 点	非单调是 PLT 特有	“算力匹配” 是实测步时，非 FLOPs

4.1 五篇论文之间的逻辑关系

五篇论文构成一个完整的论证链：

1. UT 提出机制，但参数—计算比和训练稳定性未解。
2. Huginn 解决” 能不能在现代规模训成”，代价是推理成本和知识容量。
3. Training-Free 解决” 已有模型能不能白拿收益”，并给出 ODE 解释，同时发现 MoE 需要 layer-mode。
4. LoopCoder-v2 解决” 部署时循环几次”，用 PLT 把延迟压平，发现 R=2 饱和并诊断原因。

5. Loopie 解决” 算力匹配下能不能赢”，用 layer-loop + MoE + 实测步时匹配给出肯定答案，同样选 R=2。

5. 更广的文献图谱

以下分类基于 arXiv 检索（关键词 looped/recurrent-depth/depth-recurrent transformer、latent reasoning + loop 等），保留与语言模型架构直接相关的工作。完整列表见仓库 README 与 `data/papers.csv`。

5.1 理论：表达能力、ICL、长度泛化

- Giannou et al. 2023（ICML）：循环 Transformer 是可编程计算机。
- Yang et al. 2024（ICLR）：循环模型用 <10% 参数学会与标准模型同等的学习算法。
- Gatmiry et al. 2024（ICML）：循环 Transformer 能否学会多步梯度下降；Chen et al. 2025（AISTATS）：绕过指数依赖。
- Fan et al. 2025（ICLR）：自适应步数的循环 Transformer 显著改善长度泛化。
- Saunshi et al. 2025（ICLR）：许多推理问题需要深度而非参数；k 层循环 L 次 ≈kL 层；循环模型可模拟 T 步 CoT。
- Xu & Sato 2025：形式化分离——循环擅长确定性并行（DAG 求值），CoT 擅长自约化问题的近似推断。
- Gong et al. 2025：从损失景观解释——循环架构诱导 River-V-Valley 偏置。
- Zhang et al. 2026：“预算律”：权重绑定循环模型学到的算法速度 v≈n_train/T_train，停机规则 T*=□n/v̂□。
- Wu et al. 2026：带 LayerNorm 的循环 Transformer 可证学会幂法。

5.2 规模化预训练

模型	规模	Token	循环	备注
Huginn-0125	3.5B 稠密	0.8T	r̄=32，可任选	首个现代规模；公开
Ouro 1.4B / 2.6B	稠密	7.7T	4 步 + 熵正则学习深度分配	匹敌 12B SOTA；优势来自知识操作而非存储；公开
LoopCoder-v2	7B 稠密 PLT	18T	2	代码模型；公开
Loopie-20B-A2B / 6B-A0.6B	MoE	3.5T	2（layer-loop）	算力匹配首胜；公开
Nanbeige4.2-3B	3B	28T	2（层栈复用）	agentic 产品模型，超过 Qwen3.5-9B、Gemma4-12B
SMELT	MoE 至 54B 非嵌入参数	–	中间一半层 ×2	扩展律：省 6.8–18% 训练 FLOPs

规模化预训练的共同点：**循环次数都很小（2–4）**，除 Huginn；**最新三个都是 layer-loop 或中段循环**；**MoE 骨干成为默认**。

5.3 效率：延迟、KV 内存、量化

- **延迟**：Parallel Loop Transformer（跨循环位置偏移，延迟 ≈ 单次前向）；Efficient Parallel Samplers（Geiping 2025）把循环模型视为扩散语言模型，diffusion-forcing 采样器免调参 5× 加速；Continuous Depth-wise Batching（Bae 2025）。
- **KV 内存**：Huginn 的循环 KV 共享（预算 4 不掉分）；Mixture-of-Recursions 的 KV 共享变体；MELT（跨循环共享单份 KV + 门控更新，Ouro 微调后常数内存）；**Looped Latent Attention**（发现跨循环 KV 轨迹低秩，SVD 编码器 32× 近无损，H200 上 Ouro-1.4B 批容量 32→768）；CHASE（循环 SSM 的跳出）。
- **注意力复杂度**：LT2（线性/稀疏注意力 + 循环，循环让线性注意力迭代精化记忆、让稀疏注意力扩大感受野；Ouro-hybrid-1.4B 约 1B token 转换）。
- **量化**：LoopQ（首个系统研究，W4A4 下相对最强 PTQ 基线平均精度 +68.8%）。
- **部署实录**：Halloran 2026 修复 Nanbeige4.2-3B 在 Apple Silicon 上的五个 bug，指出层复用使峰值注意力内存翻倍，分块预填充把可用上下文扩大 2.7×。

5.4 稳定性与训练理论

- Huginn 的两次失败（隐状态坍缩、循环坍缩）与三明治归一化。
- Wang et al. 2026（残差缩放）：权重共享使残差更新跨迭代相关，需要 ε=1/N 而非 1/√L；分解参数化 ε=λ/(N√L)；最优学习率只依赖唯一层数 L，可从小 N 直接迁移到大 N。
- DeepLoop：Post-LN DeepNorm 指数需从 1/4 升到 1/2，$\alpha = (2N)^{1/2}$，$\beta = (8N)^{-1/2}$。
- Fully Looped Transformer：把循环间信号分布到所有层 + 注意力注入，稳定训练到 12 次循环。

- STARS（ICML 2026）：雅可比谱半径正则化 + 随机循环采样，让潜在状态趋向渐近稳定不动点，缓解" 某深度达峰后崩溃"。
- Fixed-Point Reasoners：pre-norm + 残差缩放 + 不动点收敛作为停机。
- Think Shallow, Solve Deep：算子的有限时间动力学 regime（settling / marginal / drifting）预测加深是否安全；单一终端不动点目标同时改变 regime 与深度行为；Huginn 属于 non-settling。
- Stabilizing Extrapolation via Learned Stochastic Stopping：训练时循环次数随机化大幅降低 OOD 方差。

5.5 自适应计算与停机

- 经典：ACT（2016）、PonderNet（2021）、CALM（2022）。
- Mixture-of-Recursions（NeurIPS 2025）：轻量路由器给每个 token 分配递归深度，只对活跃 token 做注意力和 KV 缓存，135M–1.7B 形成新 Pareto 前沿。
- Think-at-Hard（ICML 2026）："潜在过思考"——多数 token 第一遍已对，后续迭代反而改错；只在 93% 之外的难 token 上循环，+3.8–4.4%。
- LoopFormer（ICLR 2026）：shortcut-consistency 训练让不同长度轨迹对齐，按预算弹性推理。
- RecurTrace（2026-09）：循环记忆注意力（沿循环时间轴回看自己的历史状态）+ oracle 监督的停机头；MathQA 平均 2.0 次循环达 56.9%，ACT/PonderNet 坍缩到 1 次。
- Adaptive Depth in Looped Transformers（Popescu）：诊断学到的停机门。
- Two-Scale Latent Dynamics：基于步长二阶差分的提前退出优于 Huginn 的 KL 准则。

5.6 改造与免训练

- Relaxed Recursive Transformers（ICLR 2025）：把 Gemma 2B 等改造为共享块 + 逐层 LoRA，恢复大部分性能。
- Retrofitted Recurrence（McLeish 2025）：课程式逐步增加循环次数，数学任务上优于直接后训练原模型。
- Training-Free Looped Transformers（2026）：零训练，中间 4 层 RK 子步。
- Program-of-Layers（ICML 2026）：学一个小网络为每个输入生成" 跳过/重复哪些层" 的程序，常以更少层数取得更高精度。
- Retrofitting Recurrent Depth（Shapiro 2026）：两个参数预算下的安装、外推、迁移、保留研究。

5.7 后训练与强化学习

- LoopCoder-v2：Thinking（显式 CoT）叠加在 R=2 循环上，LiveCodeBench +26.9，超加。
- Loopie：SPT（SFT 掩码 + PT 规模，2T token 10 epoch 不过拟合、不遗忘）+ GSPO/DAPO RL。
- RLTT（ICML 2026）：标准 GRPO 只给最终潜在状态分配 credit，与循环模型内部计算错配；沿整条潜在轨迹分配奖励，Ouro-2.6B 数学 +10.9%。
- LoopRPT：把下一 token 预测重构为推理任务，用 EMA 教师给潜在步直接分配强化信号，压缩有效推理所需迭代数。
- LOTUS（Fan 2026）：并行处理 K 个潜在块、每个位置用金标 CoT token 交叉熵监督；首个在 3B 规模弥合潜在 CoT 与显式 CoT 差距的方法，思考阶段延迟降 2.5–6.9×。
- LoopMTP：第 t 次循环的隐状态软对齐到 t 步之后的 token 嵌入，稳定训练到 15 次循环。
- Dense Supervision Is Not Enough：循环模型的" 读出盲区"。

5.8 可解释性与动力学

- Huginn 论文：轨道、滑块、不动点；路径独立。
- Lu et al. 2025：logit lens 探测 Huginn，可解释潜在 CoT 证据有限，探测结果高度依赖层与解码方式。
- Two-Scale Latent Dynamics：块内小步精化 + 块间大步漂移；训练中步长变小且趋于正交。
- Jacobian Lens（2026-09）：Ouro 每次循环重建" 全局工作区"，线性传输无法跨循环边界；Huginn 跨 16 次循环携带内容，但读写只在约 2 次循环的滑动窗内起效。
- LoopCoder-v2：有效秩在第 2 次循环达峰后下降；更新方向震荡。
- Observable Patterns Are Not Explanations：对潜在推理模型的因果—几何分析，警示把可观察模式当解释。
- Step-resolved data attribution for looped transformers。

5.9 小型递归推理器

与 LLM 路线并行的另一支：不做语言建模，用几百万到几千万参数的循环网络解结构化难题。

- HRM（2025-06）：27M 参数、1000 样本，两个不同频率的递归模块，数独/迷宫近乎完美，ARC 超过更大模型。
- TRM（2025-10）：单个 2 层 7M 网络，ARC-AGI-1 45%、ARC-AGI-2 8%，超过多数 LLM。

- Attractor Models（2026-05）：不动点隐式微分，训练内存与有效深度无关；27M 参数数独 91.4%；770M 模型超过 2× token 的 1.3B Transformer；“均衡内化” 现象——推理时可以去掉求解器。
- Denoising Recursion Models：多步去噪的递归训练，ARC-AGI 上超过 TRM。
- 批判性分析：Hierarchical Reasoning Models: Perspectives and Misconceptions；Are Your Reasoning Models Reasoning or Guessing。

5.10 跨模态与应用

- 视觉：LoopViT（18M 参数 ARC-AGI-1 65.8%，超过 73M 集成）、RD-ViT（语义分割）、ELT（视觉生成）、Déjà View（多视角 3D 重建）。
- 机器人：Recurrent-Depth VLA（单次迭代 0% 成功率的任务四次迭代 >90%，比基于 token 推理的 VLA 快 80×）。
- 语音：Depth-Conditioned Looped Transformers for ASR。
- Agent：Looped LMs Improve Compositional Tool Calling（多步工具调用随循环深度提升）；Nanbeige4.2-3B 的 agentic 部署。
- 多模态大模型：Looping Back to Move Forward（递归 LMM）。

6. 十条趋势洞察

洞察 1：循环深度跨过了” 算力匹配” 这道门，但只在窄门里

Loopie（2026-07）和 SMELT（2026-09）第一次在与普通模型**匹配训练成本**的条件下赢：前者按实测步时匹配，800B token 后反超 Qwen3-30B-A3B 复现版；后者在 FLOPs、参数、KV 三重匹配下拟合扩展律，节省 6.8–18% 训练 FLOPs，并且优势随规模不消失（SMELT 到 54B 非嵌入参数）。这两项结果把循环深度从” 参数效率技巧” 变成” 算力效率技巧”。

胜利的配置极窄：**MoE 骨干、layer-loop 或中段循环、循环两次**。Schwethelm et al. 的迭代深度扩展律解释了为什么窄：一次循环只值 $r^{0.46}$ 份独立参数（截断反传下更低到 0.38），所以用循环替代参数在 FLOPs 匹配下必输；Loopie 的胜利不是靠循环换参数，而是靠循环省下的**激活内存**换 micro-batch、再换宽度。这是一个基础设施层面的红利，理论 FLOPs 看不见它。

洞察 2：“循环几次” 分裂成两个 regime

把所有报告的最优循环次数放在一起：Loopie 2、LoopCoder-v2 2、SMELT 2、Nanbeige4.2 2、Ouro 4、Huginn 32（可到 64）、RecurTrace 平均 2.0、Think-at-Hard 在 93% 的 token 上 1。

分裂线是**算力约束在哪里**。预训练算力受限时，每多一次循环就是训练 FLOPs 乘一个系数，边际收益在 R=2 后衰减最快（Loopie Figure 6），答案是 2。推理时算力可变时，同一个模型可以在难题上多循环、易题上少循环（Huginn 的 ARC-C 随 few-shot 数从 8 次饱和到 32 次饱和），答案是” 按需、自适应”。

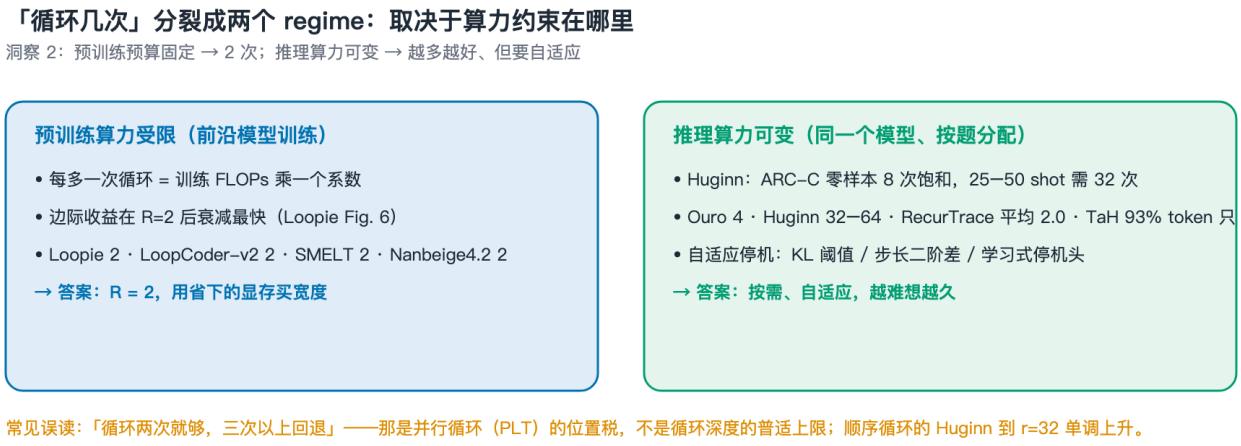


图 *2*：“循环几次” 取决于算力约束在哪里。

LoopCoder-v2 的” 三次以上回退” 经常被引用为循环深度的普适上限。论文自己的解释是 PLT 的跨循环位置偏移（CLP）在每次循环都收一笔固定的位置税，税额是第 3 次以后边际收益的 30–45 倍；顺序循环没有这一项。它是**并行化设计的代价**，不是循环的性质。

洞察 3：循环的粒度在变细，位置在向中间收敛

从 UT 的整模型循环，到 Huginn 的三段式（首尾不循环），到 Loopie/Training-Free/SMELT 的逐层或中段循环，再到 MixerLoop 只循环序列混合器而 FFN 只做一次（保留 41.5% 收益、省 45.9% 循环 FLOPs），粒度一路变细。三条独立证据指向同一个位置规则：Training-Free 的深度分数规则（最优窗口中心在相对深度 0.43–0.71，众数 0.5）、SMELT 的" 中间一半"、Huginn 的" 首尾层与中间层功能不同" 的设计依据。首尾层负责嵌入和读出、特化程度高，不宜复用；中间层同质、可复用。

对 MoE，layer-loop 还有一个硬理由：block-loop 下每次迭代路由到不同专家（routing thrash）。Training-Free（不训练）与 Loopie（从头训练）在这一点上独立一致。

洞察 4：循环是 ODE 流还是不动点迭代？两种哲学正在分化

Training-Free 论文的发现是预训练 Transformer 的中段" 在任何有用的意义上都不是压缩映射"：Anderson、heavy-ball、Aitken 等不动点加速器都不如简单的 RK 子步。因此对**已训练好的**模型，正确的循环是把 [0,1] 积分得更细（阻尼欧拉 $x \leftarrow (1-1/K)x+(1/K)g(x)$），而不是推进到 t=K。

另一支工作反过来**把循环训练成不动点迭代**：STARS 用雅可比谱半径正则化、Fixed-Point Reasoners 用不动点收敛作停机、Attractor Models 用隐式微分求均衡。它们的动机是稳定性：Viakhirev et al. 把算子分为 settling / marginal / drifting 三类，settling 算子加深不会变差，且一个终端不动点目标就能把 drifting 变成 settling；Huginn 属于 non-settling。

两种哲学对应两个场景：改造已有模型时顺着 ODE 流走；从头训练时可以把稳定性设计进目标里。LoopCoder-v2 观察到的" 更新方向震荡（$\cos\theta<0$）"正是 non-settling 的表现。

洞察 5：训练稳定性从经验教训变成了理论

Huginn 两次失败的训练（隐状态坍缩、循环坍缩）在 2025 年是经验教训，到 2026 年成了定理：残差缩放需要 $1/N$ 而非 $1/\sqrt{L}$（Wang et al.），DeepNorm 指数要从 1/4 升到 1/2（DeepLoop），最优学习率只依赖唯一层数、可跨循环次数迁移。工程上，Fully Looped Transformer 稳定训练到 12 次循环，LoopMTP 到 15 次。加上 SmallInit 与三明治归一化，"循环模型难训" 这个 2018 年以来的障碍在 2026 年基本被拆除。

洞察 6：推理成本是最后一个大瓶颈，而且正在被分解为三个子问题

循环模型的推理成本有三项：延迟（串行循环）、KV 内存（每次循环一份）、以及量化脆弱性。2025–2026 年每一项都有专门方案：延迟靠 PLT 的并行化或 Geiping 的扩散式并行采样器（5×）；KV 靠共享（Huginn、MoR、MELT）或压缩（Looped Latent Attention 发现跨循环 KV 低秩，32× 近无损）；量化靠 LoopQ。这些方案互相独立，可以叠加。

一个重要的度量口径问题：Loopie 的" 算力匹配" 是训练步时匹配，其推理时每 token 做 54 次块计算对比参考的 48 次，KV cache 更大，论文没报推理延迟。**训练算力匹配的胜利不自动等于推理成本匹配的胜利**。

洞察 7：隐式循环与显式 CoT 互补而非替代

三条证据：LoopCoder-v2 在 R=2 上叠加 thinking 模式，LiveCodeBench +26.9，远超两者单独贡献之和；LOTUS 用金标 CoT token 监督潜在位置，首次在 3B 规模追平显式 CoT 且思考延迟降 2.5–6.9×；Xu & Sato 的形式化分离说明两者各有擅长的问题类。

配套的发现是**标准 RL 与循环模型错配**：GRPO 只给最终潜在状态分配 credit（RLTT 指出这是 RL 改进循环模型屡次失败的原因），把奖励沿潜在轨迹分布后 Ouro-2.6B 数学 +10.9%；LoopRPT 进一步在预训练阶段就给潜在步分配强化信号。给循环模型做后训练需要重新设计目标，直接套用 CoT 时代的 RL 配方会浪费循环。

洞察 8：循环模型的能力剖面已经清楚——擅长" 操作" 知识，不擅长" 存储" 知识

Ouro 的对照实验把优势归因于知识操作而非知识容量；Huginn 闭卷弱、开卷强（OBQA +11.0）；Frey et al. 发现循环主要帮数学推理，记忆库主要帮常识任务，两者结合才能全面超过 iso-FLOP 基线；Dual-Path 把" 深"（循环子层）和" 宽"（大 FFN）作为并行通路，学到的门控显示标点、符号、算术 token 走深路，功能词和词汇内容走宽路。

这给出了循环模型的最佳搭档：**检索、长上下文、MoE 专家库**——任何把" 存储" 外置的机制。Loopie 与 SMELT 选 MoE 骨干，Adaptive Loops and Memory 加门控记忆库，都是这一逻辑。

洞察 9：应用面正在从数学谜题扩展到 agent 与具身智能

2025 年的循环模型证据集中在数独、迷宫、ARC、GSM8K；2026 年扩展到三类新场景：（1）**软件工程 agent**——LoopCoder-v2 的 SWE-bench Verified +21.4，Terminal-Bench 大幅提升；（2）**工具调用**——Popescu et al. 发现多步、有依赖的组合式工具调用随循环深度提升，Nanbeige4.2-3B 以 3B 参数在 agentic 基准上超过 Qwen3.5-9B；（3）**机器人**——RD-VLA 用循环动作头把某些任务从 0% 提到 >90%，比 token 推理 VLA 快 80×。共同点是这些任务都需要在一次前向内维护中间状态和依赖关系，正是循环的强项。

洞察 10：生产化已经开始，工具链还没跟上

开源的循环模型已有 Huginn、Ouro、LoopCoder-v2、Loopie、Nanbeige4.2-3B 五个家族，其中 Nanbeige4.2-3B 用 28T token 训练并作为产品发布。但 Halloran 的复现报告显示该模型在 Hugging Face transformers 上开箱即有五个 bug（包括被静默置零的 RoPE 缓冲区和已移除的 cache API 调用），修复后仍因层复用导致注意力内存翻倍。推理框架、量化工具、KV 管理都还没有为循环模型建立标准接口。Loopie 随论文发布了 megatron-loopie 与 vllm-loopie 两个分支，是往这个方向走的信号。

7. 关于 GPT-6 Astra 的推断（明确标注为推断）

以下判断基于本报告梳理的公开文献，不基于任何关于 Astra 的第一手信息。

1. 如果 Astra 确实使用 recurrent depth，公开文献中最接近” 前沿规模 + 算力匹配” 的配方是 Loopie/SMELT 路线：MoE 骨干、中段或逐层循环、循环次数很小。整模型循环 32 次的 Huginn 路线在前沿预训练预算下没有算力匹配的证据。
2. “OpenAI 找到了绕过两次饱和的办法” 这一常见推测的前提不成立——两次饱和是 PLT 特有的现象。顺序循环模型（Huginn）本身就没有这个瓶颈；真正的瓶颈是推理成本，而 KV 压缩（32×）、扩散式并行采样（5×）等公开方案已经把它压低了一个量级。
3. The Information 报道提到的” 安全担忧” 与文献中的一个真实问题吻合：潜在推理不产生可读的中间 token，可解释性工作（Lu 2025、Jacobian Lens 2026）显示循环模型内部状态的探测结果依赖层与解码方式，监督其推理过程比监督 CoT 更难。
4. 时间点上，Loopie（7 月）、Looped Latent Attention（7 月）、SMELT（9 月 1 日）与 Astra（9 月初）密集出现，更合理的解读是这个方向在 2026 年上半年同时被多个团队推到可用，而不是某一篇论文直接导致了某个产品。

8. 未解问题

1. **多循环的算力匹配**。所有算力匹配的胜利都在 R=2。R≥4 在预训练算力匹配下能否赢，没有公开证据。
2. **推理成本的完整账**。需要同时报告训练步时、推理延迟、KV 峰值内存的对照；目前没有一篇论文三者齐全。
3. **循环模型的扩展律**。 SMELT 拟合到 54B 非嵌入参数、Schwethelm 到 ~1B。两者的函数形式（Chinchilla 式加 r^{φ} 项）是否在 100B+ 上成立未知。
4. **自适应停机在预训练规模上的收益**。 MoR、TaH、RecurTrace 的结果都在 ≤8B；Ouro 的熵正则深度分配是唯一在 7.7T token 上验证过的。
5. **潜在推理的可监督性**。 LOTUS 证明用金标 CoT 监督潜在位置有效，但这要求 CoT 数据；无 CoT 数据时如何让潜在轨迹可读、可审计，是安全层面的开放问题。
6. **layer-loop 的机制**。Loopie 给出的三个理由（扩展性、基础设施、共享局部性）中，只有扩展性有实验；SMELT 的” 第二次访问削弱注意力沉降” 是目前唯一的机制性解释。
7. **循环与序列长度扩展的取舍**。Hidden Decoding（WeLM 617B）走序列维度加算力并声称与流水线并行更兼容；Loopie 的 layer-loop 恰恰为兼容流水线并行而设计。两条路线在 100B+ 规模上尚无正面对比。

9. 结论

八年里，循环深度从一个解决 bAbI 过拟合的编码器—解码器技巧，变成了前沿模型架构的候选组件。2026 年上半年的三个转折——免训练循环让已有模型白拿 2 个点、PLT 让延迟不随循环增长、Loopie/SMELT 让循环 MoE 在算力匹配下第一次赢——共同把它推过了工程可行的门槛。剩下的问题集中在推理成本的完整核算、多循环的算力匹配、以及潜在推理的可监督性。对研究者，最有价值的空白是 R≥4 的算力匹配扩展律与循环专用的 RL 目标；对工程团队，最实用的结论是：中段循环、两次、layer-loop、MoE，并在部署前实测延迟和 KV。

附录 A：本报告核对过的传播误差

常见说法	原文事实
Huginn “同样 0.8T token” 下循环模型碾压基线	非循环基线只训了 0.18T；同为 0.18T 时循环模型 ARC-E 53.62 vs 46.42
“循环两次就够，三次以上回退” 是循环 Transformer 的规律	是 PLT（跨循环位置偏移）特有的现象；Huginn 到 r=32 单调上升
Loopie 与 Qwen3-30B-A3B “相同算力”	相同实测优化器步时；Loopie 名义 FLOPs 高 42%，推理成本未报告
Training-Free 让开源模型” 变强”	收益约 1–2.6 个百分点，集中在知识型多选题，小蒸馏模型可能掉分
Huginn “3.5B 打出 50B 的表现”	FLOPs 等效，非基准等效；基准上接近 OLMo-7B、落后 OLMo-2

附录 B：方法与数据来源

- 五篇核心论文：arXiv 原文 PDF（`papers/original/`）与 arXiv HTML 全文（`papers/src/`），逐节核读。
- 中译：SuperTranslate 保版式翻译 + 对象级 QA（`papers/zh/`，QA 记录见 `QA_SUMMARY.md`）。
- 相关文献：arXiv API 检索（2026-09-06），关键词组合 looped/recurrent-depth/depth-recurrent transformer、latent reasoning □ (loop|recurrence|recursive)、命名模型；去重后保留 60 篇摘要于 `data/related_abstracts.json`。
- 写作规范：主张先行、必要限定只写一次、数字随处可核对；不引用无法访问的一手材料。

附录 C：术语小词典

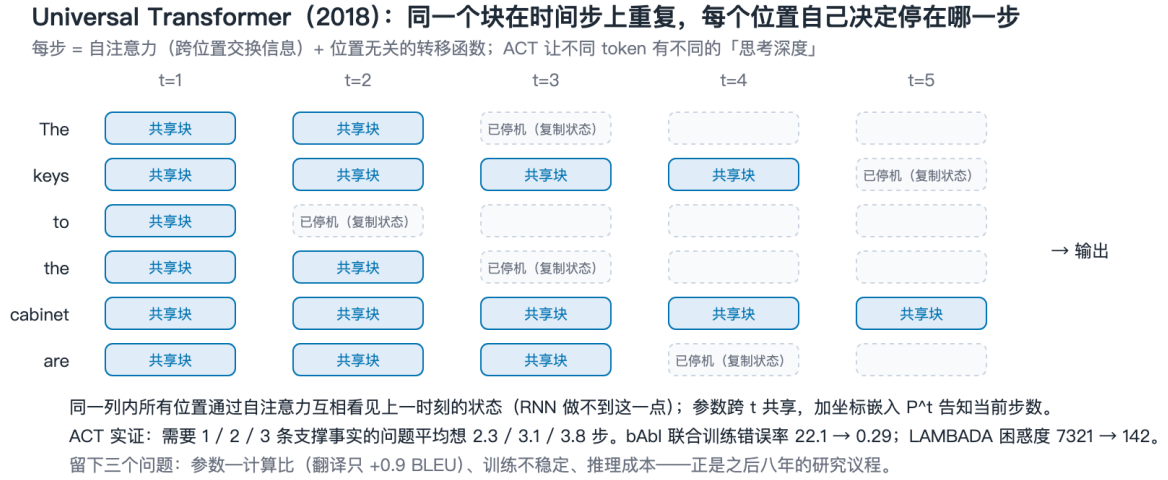
术语	一句话解释
循环深度（recurrent depth）/ 循环 Transformer	同一组参数在深度方向上重复使用；有效深度 = 存储层数 × 循环次数，参数量不变
model-loop / block-loop / layer-loop	整个层栈循环 / 只循环中间一段 / 每层各自重复几次再下传（图 1）
输入注入	每一次循环都把原始输入的嵌入重新喂给循环块，否则迭代不稳定
截断反传	只对最后 k 步循环计算梯度，让显存与循环次数无关
自适应停机（ACT）	让每个 token 自己决定循环几次，难的多算、简单的早停
路径独立	从不同随机初始状态出发，循环最终收敛到相同行为
KV cache	注意力缓存的键值；循环模型每循环一次多一份，是显存瓶颈
MoE（混合专家）	每个 token 只激活一小部分参数；“20B-A2B” = 总参数 20B、激活 2B
FLOPs 匹配 vs 实测算力匹配	按理论计算量对齐 vs 按实测训练步时对齐——循环省下的激活显存只在后者里体现
潜在推理 vs 显式 CoT	在隐空间里多算几步 vs 把中间步骤写成文字 token
不动点	再循环一次输出也不变的状态；一派工作把循环训成朝不动点收敛以保证稳定
PLT / CLP	并行循环 Transformer / 跨循环位置偏移——上一轮状态右移一位，使不同轮次并行执行
扩展律中的 φ	一次循环值多少份独立参数的指数：φ=1 完全等价，φ=0 没有容量增益；实测 0.46

解读 01 · Universal Transformers：循环深度的源头

项目	内容
论文	Universal Transformers
arXiv	1807.03819（2018-07），ICLR 2019
作者	Mostafa Dehghani*、Stephan Gouws*、Oriol Vinyals、Jakob Uszkoreit、Łukasz Kaiser（阿姆斯特丹大学 / DeepMind / Google Brain）
代码	tensor2tensor
本仓库	原文 `papers/original/1807.03819.pdf` · 中译 `papers/zh/1807.03819.zh.pdf`

读前速览

> **问题** · Transformer 深度固定，也没有 RNN 那种” 把同一个变换反复用” 的归纳偏置，在算法任务和多跳推理（bAbI）上直接过拟合。
> **做法** · 把 N 层不同参数换成同一层重复 T 次，加” 位置 + 步数” 坐标嵌入；ACT 让每个位置自己决定停在第几步。
> **结果** · bAbI 联合训练错误率 22.1 → 0.29，LAMBADA 困惑度 7321 → 142，WMT14 翻译 +0.9 BLEU。
> **局限** · 参数—计算比差（参数主导的任务收益小）、训练不稳、没有讨论推理成本——这三点成了之后八年的研究议程。
> **大白话** · 让同一个” 脑回路” 多转几圈：难的词多转几圈，简单的词转一圈就停。



图：同一个共享块沿时间步重复，每个位置在不同步数停机（示意）。

一句话

把 Transformer 的 N 层不同参数换成同一组参数重复 T 次，再给每个位置配一个” 想多久” 的停机机制（ACT），得到一个时间上并行、深度上循环、理论上图灵完备的序列模型；它在 bAbI、LAMBADA、算法任务和 WMT14 翻译上全面超过同规模 Transformer。

1. 要解决的问题

2018 年的 Transformer 有两个当时就被指出的缺陷。第一，深度固定：无论输入多难，每个 token 都经过同样的 6 层。第二，缺少 RNN 那种” 反复应用同一个变换” 的归纳偏置，因此在需要迭代的算法任务上（复制、反转、加法）表现很差，在 bAbI 这类需要多跳推理的任务上直接过拟合（作者注明” 换了多组超参和网络规模，始终过拟合”）。

Neural GPU、Neural Turing Machine 这类模型有循环和通用计算能力，但在真实语言任务（翻译）上无法与 Transformer 竞争。UT 的目标是把两边的优点合到一个模型里。

2. 核心方法

2.1 深度方向的循环

编码器从输入嵌入 $H^0 \in \mathbb{R}^{m\times d}$ 出发，每个时间步 t 对所有 m 个位置并行做两件事：先用多头自注意力交换位置间的信息，再用一个位置无关的转移函数（Transition）逐位置变换：

$$A^t = \mathrm{LN}\big((H^{t-1}+P^t) + \mathrm{MHSA}(H^{t-1}+P^t)\big), \qquad H^t = \mathrm{LN}\big(A^t + \mathrm{Transition}(A^t)\big)$$

T 步之后输出 H^T。所有时间步共享同一组注意力和转移函数参数，因此” 层数” 不再等于” 参数量”。转移函数有两种选择：逐位置的两层前馈网络，或深度可分离卷积（后者在算法任务上更好）。

2.2 坐标嵌入：位置 + 时间步

因为参数共享，模型必须知道自己” 现在是第几步”。UT 把正弦位置编码推广为二维坐标嵌入 P^t，对位置 i 和时间步 t 分别计算正弦编码再相加（式 6–7），在每一步重新加到状态上。这是后来所有循环模型里” 步数条件化”（timestep encoding）的雏形。

2.3 动态停机（ACT）

把 Graves (2016) 的 Adaptive Computation Time 装到每个位置上：每步为每个位置预测一个停机概率，累计超过阈值就停止更新、状态原样复制到后续步，直到所有位置停机或达到最大步数。这样不同 token 得到不同的” 思考深度”（ponder time）。

2.4 计算能力

固定 T 时，UT 就是一个跨层权重绑定的 Transformer。让 T 随输入长度变化，UT 可以精确还原 Neural GPU（把自注意力设为恒等、转移函数设为卷积、T = 输入长度），因此在有足够内存的前提下是图灵完备的。作者强调这与 RNN 的关键区别：RNN 的循环步只能访问固定大小的状态向量，UT 的每一步都能通过注意力访问上一步所有位置的表示。

3. 关键实验与数字

任务	Transformer	UT	UT + 动态停机
bAbI 10K，20 任务联合训练（平均错误率 / 失败任务数）	22.1 (12/20)	0.47 (0/20)	**0.29 (0/20)**
bAbI 1K，联合训练	26.8 (14/20)	8.50 (8/20)	**7.78 (5/20)**
主谓一致（5 个干扰名词）	0.883	0.892	**0.907**
LAMBADA 语言建模测试集困惑度	7321	319（固定 6 步）	**142**
WMT14 En-De BLEU（base 规模）	28.0	**28.9**	–
Copy / Reverse / Addition（训练长 40，测试长 400）字符准确率	0.53 / 0.13 / 0.07	**0.91 / 0.96 / 0.34**	–
Learning-to-Execute 记忆任务（Copy/Double/Reverse）序列准确率	0.63 / 0.55 / 0.26	**1.0 / 1.0 / 1.0**	–

三个值得单独说的观察：

1. **停机步数跟随任务难度。** bAbI 中需要 1、2、3 条支撑事实的问题，平均 ponder time 分别为 2.3±0.8、3.1±1.1、3.8±2.2。模型确实按需分配计算。
2. **动态停机是一种正则化。** LAMBADA 上动态停机模型平均走 8.2 步；作者补训了固定 8 步和 9 步的 UT，两者都好于 6 步，但都不如动态版本。多走步数本身不是全部原因。
3. **干扰越多、优势越大。** 主谓一致任务里，UT+ACT 相对当时最佳模型的优势随干扰名词数量单调增加（从 0 到 +0.027）。循环带来的收益集中在难样本上，这个模式在七年后的 Huginn 论文里以几乎相同的形式重现。

4. 为什么重要

UT 定义了这条技术路线此后所有工作的三个基本要素：**参数共享的深度循环**、**每步重新注入的条件信号**（这里是坐标嵌入，后来演变为输入注入）、**自适应停机**。今天 recurrent depth 文献中的 model-loop（整模型循环）就是 UT 的原始形态。

同时它也留下了三个长期没解决的问题，构成后续七年的研究议程：

- **参数—计算比**。共享参数让同等隐藏维度下的参数量骤减，在语言建模这种" 参数主导" 的任务上吃亏。翻译任务上 UT base 只比 Transformer base 高 0.9 BLEU，说明在参数充足的场景里循环的边际收益有限。这个问题直到 MoEUT（2024）、Loopie（2026）把循环和 MoE 结合才真正被处理。
- **训练不稳定**。 ACT 的停机概率需要额外的正则项且对超参敏感；深度循环的梯度也不好训。后续 Huginn 用截断反传、Loopie 用 layer-loop、多篇 2026 年论文用残差缩放理论来解决。
- **计算成本**。每多循环一步就多一次完整前向。UT 没有讨论推理时的延迟和 KV cache 问题；这是 PLT、MELT、Looped Latent Attention 等 2025–2026 年工作的主题。

5. 局限

- 实验规模小（最大 WMT base），没有在参数量超过 1 亿的设定下验证。
- ACT 的收益不稳定：翻译实验里作者没有使用 ACT，只用固定步数。
- 编码器—解码器结构；decoder-only 时代需要重新设计（Huginn 的 prelude–core–coda 三段式是最直接的继承者）。

6. 与本方向其他工作的关系

- **前驱**：Neural GPU（Kaiser & Sutskever 2016）、Neural Turing Machine（Graves 2014）、ACT（Graves 2016）。
- **同期**：ALBERT（Lan et al. 2019）把跨层参数共享当作压缩手段用在 BERT 上，思想相同、目的不同。
- **直接继承**：Sparse Universal Transformer（Tan et al. 2023）、MoEUT（Csordás et al. 2024）解决参数—计算比；Giannou et al. (2023) 从" 可编程计算机" 角度重新证明循环 Transformer 的通用性；Geiping et al. (2025) 把它做到 3.5B 规模。

7. 对从业者的启示

- 循环深度的收益集中在" 需要迭代才能解" 的任务上（多跳、算法、长依赖），对纯记忆型任务几乎没有帮助。评估循环模型时，应该把基准按这两类拆开看。
- 自适应停机不只是省算力，它还是正则化器。2026 年的 RecurTrace、Think-at-Hard 等工作重新验证了这一点。
- 步数条件化（告诉模型现在是第几步）与" 路径独立" 目标之间存在张力：Huginn 论文明确指出加入步数输入会破坏外推能力。UT 时代的做法并非在所有设定下都对。

附：关键数字速查

- 模型：编码器—解码器，参数跨时间步共享；转移函数为 FFN 或深度可分离卷积。
- bAbI 10K 联合训练错误率：22.1% → 0.29%。
- LAMBADA 测试困惑度：7321 → 142。
- WMT14 En-De：28.0 → 28.9 BLEU。
- ACT 平均步数与所需事实数：2.3 / 3.1 / 3.8（1 / 2 / 3 条事实）。

术语小词典

术语	一句话解释
循环深度（recurrent depth）/ 循环 Transformer	同一组参数在深度方向上重复使用；有效深度 = 存储层数 × 循环次数，参数量不变
model-loop / block-loop / layer-loop	整个层栈循环 / 只循环中间一段 / 每层各自重复几次再下传；见 `assets/figures/diagrams/loop_granularity.png`
输入注入（input injection）	每一次循环都把原始输入的嵌入重新喂给循环块，否则迭代不稳定
截断反传（truncated backprop）	只对最后 k 步循环计算梯度，让显存与循环次数无关
自适应停机（ACT / adaptive halting）	让每个 token 自己决定循环几次，难的多算、简单的早停
路径独立（path independence）	从不同的随机初始状态出发，循环最终收敛到相同的行为
KV cache	注意力缓存的键值；循环模型每循环一次都会多一份，是显存瓶颈
MoE（混合专家）	每个 token 只激活一小部分参数（专家）；“20B-A2B” = 总参数 20B、激活 2B

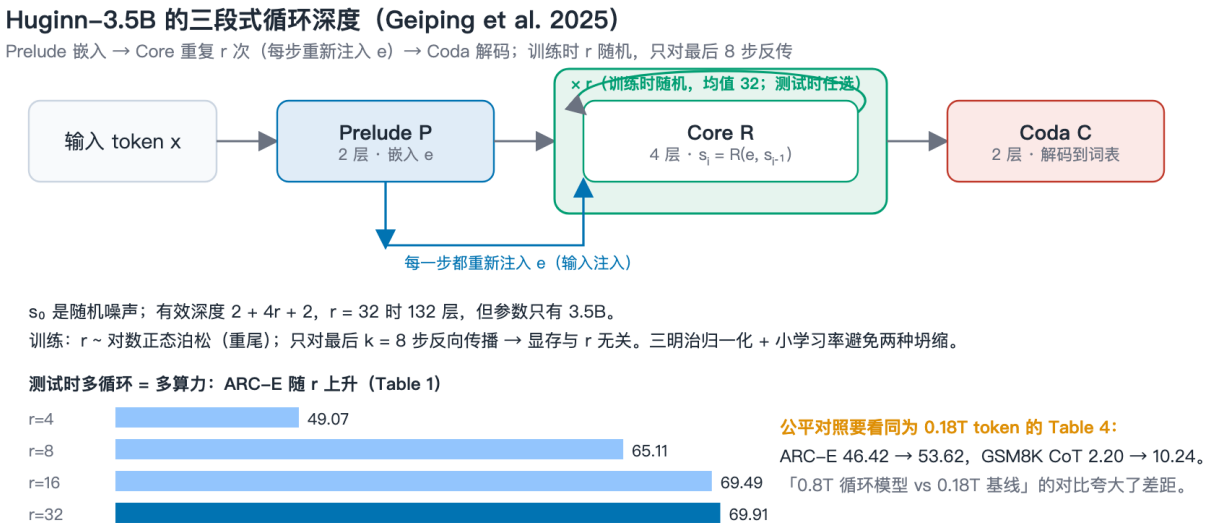
术语	一句话解释
FLOPs 匹配 vs 实测算力匹配	前者按理论计算量对齐，后者按实测训练步时对齐——循环模型省下的激活显存只在后者里体现
潜在推理 vs 显式 CoT	在隐空间里多算几步 vs 把中间步骤写成文字 token
不动点（fixed point）	再循环一次输出也不变的状态；一派工作把循环训成朝不动点收敛以保证稳定
PLT / CLP	并行循环 Transformer / 跨循环位置偏移——把上一轮状态右移一位，使不同轮次能并行执行

解读 02 · Scaling up Test-Time Compute with Latent Reasoning：把” 循环深度” 做到 3.5B

项目	内容
论文	Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach
arXiv	2502.05171（2025-02）
作者	Jonas Geiping、Sean McLeish、Neel Jain、John Kirchenbauer、Siddharth Singh、Brian R. Bartoldson、Bhavya Kailkhura、Abhinav Bhatele、Tom Goldstein（ELLIS Tübingen / MPI-IS / 马里兰大学 / LLNL）
模型 / 代码	huginn-0125 · recurrent-pretraining
本仓库	原文 `papers/original/2502.05171.pdf` · 中译 `papers/zh/2502.05171.zh.pdf`

读前速览

> 问题 · 2018 年就有的” 循环深度” 想法，能不能在现代 LLM 规模上训成，并且真的随推理时循环次数提升？
> 做法 · 2 层序幕 + 4 层循环核心 + 2 层尾声；训练时每条样本随机采样循环次数（均值 32），只对最后 8 步反传；800B token、4096 块 GPU。
> 结果 · 循环次数从 4 提到 32：ARC-E 49.1 → 69.9，GSM8K CoT 近 0 → 42.1；同为 0.18T token 的公平对照 ARC-E +7.2、GSM8K 约 5 倍。
> 局限 · 每多一次循环多一次核心块前向；事实记忆弱（MMLU 31）；只有一次大规模训练、学习率没冷却。
> 大白话 · 参数不多，但可以在” 脑子里” 多想几遍再开口；题越难，多想越有用。



图：三段式架构与训练要点；右下为 *ARC-E* 随循环次数上升，以及公平对照的正确读法。

一句话

在 4096 块 AMD MI250X 上用 800B token 从头预训练一个 3.5B 参数、“2 层序幕 + 4 层循环核心 + 2 层尾声” 的 decoder-only 模型（Huginn-0125），训练时随机采样循环次数、只对最后 8 步反传；推理时把循环次数从 4 提到 32，ARC-E 从 49.1 涨到 69.9、GSM8K CoT 从近 0 涨到 42.1，且不需要任何思维链数据。这篇论文给”recurrent depth” 这个词定了名，并首次证明它能在现代语言模型规模上训练成功。

1. 要解决的问题

主流的测试时扩展（o1、R1）靠生成更长的思维链，代价是：需要专门构造的长 CoT 训练数据、上下文窗口要很大、而且所有中间思考都必须被压成离散 token。作者想验证另一条路：让模型在连续隐空间里迭代” 思考”，通过重复应用同一个块把网络展开到任意深度。

这条路 2018 年 UT 就提过，但从未在现代 LLM 规模上训成。本文要回答的是工程问题：怎样的架构、初始化、训练目标能让深度循环在数十亿参数、近万亿 token 的设定下稳定收敛，并且真的随循环次数提升性能。

2. 核心方法

2.1 三段式架构：Prelude → Core（循环）→ Coda

设循环次数为 r，输入 token 序列 $\mathbf{x}$：

$$\mathbf{e} = P(\mathbf{x}), \qquad \mathbf{s}_0 \sim \mathcal{N}(0, \sigma^2 I), \qquad \mathbf{s}_i = R(\mathbf{e}, \mathbf{s}_{i-1})\ (i = 1..r), \qquad \mathbf{p} = C(\mathbf{s}_r)$$

- **Prelude** P（2 层）把 token 嵌入到隐空间。
- **Core** R（4 层）接收上一步状态 $\mathbf{s}_{i-1}$ 与嵌入 $\mathbf{e}$ 的拼接，经一个适配矩阵 $A : \mathbb{R}^{2h} \to \mathbb{R}^h$ 映回隐维度，再过 4 层 Transformer。**每一步都重新注入 $\mathbf{e}$**——作者用梯度下降类比：优化 $E(\mathbf{x}, \mathbf{y})$ 时每一步都要用到数据 $\mathbf{y}$，只在第一步喂入数据的迭代不可能稳定。初始状态 $\mathbf{s}_0$ 是随机噪声，配合输入注入，模型收敛到与初始化无关的稳态（路径独立）。
- **Coda** C（2 层）解码到词表。

主模型形状 $(l_P, l_R, l_C) = (2, 4, 2)$，隐维度 $h = 5280$，55 个头 × 96 维，MLP 内维 17920。参数分布：非循环部分 1.5B、循环核心 1.5B、绑定嵌入 0.5B，合计 3.5B。$r = 32$ 时有效深度 $2 + 4 \times 32 + 2 = 132$ 层。

2.2 微观设计：三明治归一化

每层用四个 RMSNorm 把注意力和 MLP 分别” 夹住”（n_1–n_4）。小规模下 pre-norm/post-norm 都能训，大规模下只有这种三明治格式能训出真正使用循环的模型（见第 4 节两次失败的训练）。其余组件常规：RoPE（base 50000）、门控 SiLU MLP、QK 偏置。

2.3 训练目标：随机展开 + 截断反传

对每个训练样本随机采样循环次数 $r \sim \Lambda$，优化期望损失：

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}}\, \mathbb{E}_{r \sim \Lambda}\, L(m_\theta(\mathbf{x}, r), \mathbf{x}')$$

Λ 取对数正态泊松分布：$\tau \sim \mathcal{N}(\log \bar{r} - \frac{1}{2}\sigma^2, \sigma)$，$r \sim \text{Poisson}(e^\tau) + 1$，均值 $\bar{r} = 32$，$\sigma = 1/2$。它大多采到小于 32 的值，但有重尾，偶尔采到很大的 r。

只对最后 $k = 8$ 步反向传播。这样激活内存与反传计算和 r 无关，才能承受重尾分布。Prelude 因为其输出 $\mathbf{e}$ 每步都被注入，仍然在每一步都拿到梯度。这是 RNN 里截断 BPTT 在深度方向上的翻版。

2.4 规模化训练的工程细节

- 数据：800B token（实际 795B），公开数据，偏向代码与数学；直接把指令数据混入预训练。词表 65536，序列长 4096。
- 硬件：ORNL Frontier，4096 块 MI250X，21 段每段最多 12 小时；单节点 108.75 TFLOP/s（87% 可达利用率），4096 卡时 52–64 TFLOP/s。因为参数共享，即使最大模型也只用数据并行加优化器分片，不需要张量并行。
- 优化：Adam（0.9, 0.95），恒定学习率 4×10^{-5}，4096 步预热，无衰减；每个 micro-batch 同步采样同一个 r（锁步采样）避免 worker 空转。
- 初始化：Takase et al. (2024) 方案，方差 $2/(5h)$；输出投影方差 $1/(5hl)$，其中 $l = 132$ 为有效层数。

3. 关键实验与数字

3.1 性能随循环次数单调上升（Table 1）

循环次数 r	ARC-E	ARC-C	HellaSwag	MMLU	OBQA	PiQA	SciQ	WinoGrande
4	49.07	27.99	43.46	23.39	28.20	64.96	80.00	55.24
8	65.11	35.15	58.54	25.29	35.40	73.45	92.10	55.64
16	69.49	37.71	64.67	31.25	37.60	75.79	93.90	57.77
32	**69.91**	**38.23**	**65.21**	**31.38**	**38.80**	**76.22**	93.50	**59.43**

横向比较：与 OLMo-7B（2.5T token）接近，落后于 OLMo-2（4T token、精心清洗的数据）。作者自己的定位是” 把 FLOPs 用得像 32B 模型的 3.5B 模型”：预训练每 token 的原始 FLOPs 接近 32B 稠密模型，测试时继续加循环可到约 50B 模型的等效算力。

3.2 数学与代码（Table 2、3）

GSM8K CoT 34.80/42.08（带系统提示，flexible/strict）、Minerva MATH 12.58、MBPP 24.80、HumanEval 23.17。数学上超过除 OLMo-2 外所有对比模型；代码上超过所有通用开源模型。恒定学习率意味着可以用权重平均模拟退火：EMA 模型在 $r = 64$ 下 GSM8K flexible 提升到 47.23%。

3.3 公平对照：同数据同设定的非循环基线（Table 4）

这是全文最重要的表，也是被二次传播时最常引错的表。基线是**同架构、只过一次核心块、训练 180B token** 的模型；循环模型也有一个 180B 的早期检查点。

模型	Token	ARC-E	HellaSwag	GSM8K CoT
固定深度基线	0.18T	46.42	37.34	1.82/2.20
循环模型早期检查点，$r = 32$	0.18T	53.62	48.80	9.02/10.24
循环模型早期检查点，$r = 1$	0.18T	34.01	29.19	0.00/0.15
循环模型最终，$r = 32$	0.8T	69.91	65.21	34.80/42.08
循环模型最终，$r = 1$	0.8T	34.89	29.34	0.00/0.00

正确的读法：**同为 180B token**，循环模型比基线 ARC-E 高 7.2 点、HellaSwag 高 11.5 点、GSM8K 高约 5 倍。某些科普文章把 0.8T 的循环模型和 0.18T 的基线并列成” 同样 0.8T” 来比，夸大了差距；论文本身没有 0.8T 的非循环基线。

另一个信息藏在 $r = 1$ 那两行：从 180B 到 800B，$r = 1$ 的成绩几乎不动（34.01 → 34.89）。也就是说后 620B token 学到的能力**全部编码在循环块的迭代里**，序幕和尾声没有变强。

3.4 循环在哪里最有用

- 难任务受益大（ARC-C）、纯事实回忆受益小（SciQ）。
- **上下文越多，需要的循环越多。** ARC-C 零样本时 8–12 次饱和，1-shot 约 20 次，25–50 shot 需要 32 次（Figure 9）。模型用更多的循环从更多的上下文里提取信息。
- **开卷 vs 闭卷。** OBQA 提供相关事实后，Huginn 从 38.2 提到 49.2（+11.0），几乎追上 OLMo-2。结论：循环模型记的事实少，但对上下文推理的能力强。

4. 两次失败的训练：为什么规模化这么难（Section 4.3）

- **失败 1**：无参数 RMSNorm、无嵌入缩放、适配器用加法 $A(\mathbf{s}, \mathbf{e}) = \mathbf{s} + \mathbf{e}$、峰值学习率 4×10^{-4}。**隐状态坍缩**：序列内 token 间隐状态相关度迅速到 1.0，所有位置预测同一个状态。每一次循环都在增加 token 间的相关性，直到坍缩。
- **失败 2**：加嵌入缩放、改回常规 pre-norm、换成学习式适配器。相关度前 150 步冲到 1.0 后恢复，但**循环坍缩**：$r = 1$ 与 $r = 32$ 的验证困惑度完全一样——模型早早学会忽略输入状态 $\mathbf{s}$，掉进一个不用循环的局部最优。
- **成功**：回到三明治归一化，学习率再降 10 倍到 4×10^{-5}。之后 750B token 平稳无 loss spike。

这两条经验被后来的 Loopie（SmallInit）、DeepLoop、“On the Residual Scaling of Looped Transformers”（$\varepsilon = 1/N$ 缩放）系统化为理论。

5. 循环深度带来的” 免费” 能力（Section 6）

这些能力在标准 Transformer 上都需要专门训练，在 Huginn 上零样本可用：

1. **逐 token 自适应算力。**以相邻两步输出分布的 KL 散度小于 5×10^{-4} 为停机条件。MMLU 各类别所需步数不同：高中数学最少，道德情境平均多 3.5 步。MT-Bench 5.63 → 5.56，几乎无损。
2. **KV cache 共享。**每个 token 只保留 k 个循环步的 KV，第 i 步读写第 $i \bmod k$ 个槽位。预算 4 时 MT-Bench 5.86，不掉分。原因是所有循环步的 KV 由同一组投影矩阵产生，天然” 对得上”。
3. **连续思维链。**不重新随机初始化 $\mathbf{s}_0$，而是用上一个 token 的最终状态 $\mathbf{s}_r$ 热启动，平均少走 1–2 步。这与 Coconut（Hao et al. 2024）的思想相通，区别在于 Huginn 从头预训练循环、不需要 CoT 数据。
4. **自推测解码。**用少量循环起草、多循环验证，起草时算出的状态可以在验证时复用，不需要单独的 draft 模型。

6. 隐空间里发生了什么（Section 7）

对轨迹 $\{\mathbf{s}_i\}$ 做 PCA（Figure 11–12）：

- 多数 token 单调收敛到一个不动点。
- 数值推理相关的 token（GSM8K 题目里的”3”）进入**多维轨道（orbit）**，在三对主成分方向上都呈周期运动；结构性 token（“makes”、“thinks”）也常见轨道。
- 个别关键 token（“wrong”）表现为**滑块（slider）**：沿单一方向持续漂移，可能在实现计步。
- 收敛速度依赖上下文：同一句里三个相同的”.” 收敛行为不同；问题的关键部分和回答起始处” 深思” 最久。
- 路径独立成立：换不同的 $\mathbf{s}_0$，轨道、不动点、漂移模式相同。

这些几何模式不在训练目标里，是规模化训练后自发出现的。作者的解读是模型在利用高维空间” 空间化地” 组织计算，而不是像文字 CoT 那样线性排列。

7. 局限

- 只训了一次大规模实验：47000 步、学习率从未冷却、数据配比未消融。作者自称 proof-of-concept。
- 每多循环一步就多一次核心块前向，$r = 32$ 的推理成本是 8 层稠密模型的 16 倍以上；KV cache 也随 r 线性增长（第 5 节的共享是缓解而非解决）。
- 事实记忆能力弱（MMLU 31.4、闭卷 OBQA 低），与” 参数少、算力多” 的设计取向一致。
- Lu et al. (2025) 用 logit lens 探测 Huginn，只发现有限的可解释” 潜在 CoT” 证据，且增加循环的边际收益递减。Viakhirev et al. (2026) 把 Huginn 归入” 非稳定（non-settling）“动力学类别——更多循环不保证答案不变。

8. 与本方向其他工作的关系

- 思想源头 UT（2018）、deep thinking 系列（Schwarzschild 2021、Bansal 2022：随机展开 + 输入注入）、路径独立均衡模型（Anil 2022）。
- 与 DEQ 的区别：DEQ 直接求不动点，Huginn 用截断展开训练，作者认为后者是更可扩展的替代方案。
- 直接后继：Ouro（2.6B，7.7T token，4 步循环）、Loopie（MoE，2 步 layer-loop）、LoopCoder-v2（PLT，2 步）；Geiping 团队后续的并行采样器（2510.14961）把循环模型与扩散语言模型联系起来，实现 5× 加速；McLeish et al. (2025) 研究如何把已有的非循环模型改造成循环模型。
- 论文第 9 节的预测已被验证：循环（擅长推理）与 MoE（擅长存储）互补，“未来的架构会同时包含两者”——Loopie 与 SMELT 正是这个方向。

9. 对从业者的启示

- 归一化和初始化在小规模上看不出差别，在大规模上决定成败。任何循环架构的小规模消融都要对” 是否真正使用了循环”（比较 $r = 1$ 与 $r = \max$ 的困惑度）做检查。
- 输入注入 + 随机初始状态 + 随机循环次数 + 截断反传，是目前训练可外推循环模型的标准配方。
- 循环模型的强项是” 用算力换推理”，弱项是” 用参数存知识”。部署时给它检索或长上下文，收益比给稠密模型大。
- 论文里的”3.5B 打出 50B” 指的是 FLOPs 等效，不是基准分数等效。与 OLMo-2-7B 的差距是真实存在的。

附：关键数字速查

- 形状 (2,4,2)，$h = 5280$，3.5B 参数，$r = 32$ 时 132 层有效深度。
- 训练 800B token，4096 MI250X，恒定学习率 4×10^{-5}，$\bar{r} = 32$，截断反传 $k = 8$。
- r: 4→32 时 ARC-E 49.07→69.91，HellaSwag 43.46→65.21，GSM8K CoT（strict）→42.08。
- 同 180B token 公平对照：ARC-E 46.42 vs 53.62；GSM8K CoT 2.20 vs 10.24。
- 自适应停机阈值 KL $< 5 \times 10^{-4}$；KV 共享预算 4 时 MT-Bench 不掉分。

术语小词典

术语	一句话解释
循环深度（recurrent depth）/ 循环 Transformer	同一组参数在深度方向上重复使用；有效深度 = 存储层数 × 循环次数，参数量不变
model-loop / block-loop / layer-loop	整个层栈循环 / 只循环中间一段 / 每层各自重复几次再下传；见 `assets/figures/diagrams/loop_granularity.png`
输入注入（input injection）	每一次循环都把原始输入的嵌入重新喂给循环块，否则迭代不稳定
截断反传（truncated backprop）	只对最后 k 步循环计算梯度，让显存与循环次数无关
自适应停机（ACT / adaptive halting）	让每个 token 自己决定循环几次，难的多算、简单的早停
路径独立（path independence）	从不同的随机初始状态出发，循环最终收敛到相同的行为
KV cache	注意力缓存的键值；循环模型每循环一次都会多一份，是显存瓶颈

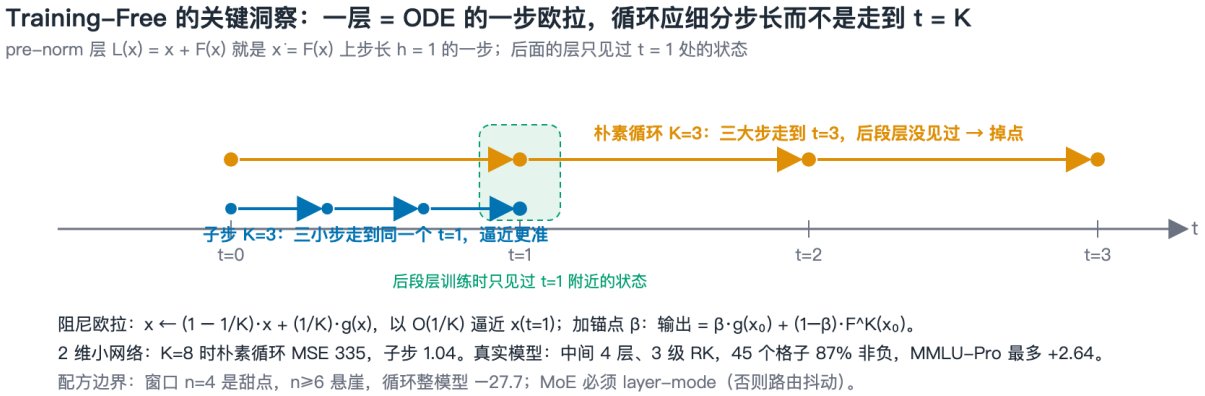
术语	一句话解释
MoE（混合专家）	每个 token 只激活一小部分参数（专家）；“20B-A2B” = 总参数 20B、激活 2B
FLOPs 匹配 vs 实测算力匹配	前者按理论计算量对齐，后者按实测训练步时对齐——循环模型省下的激活显存只在后者里体现
潜在推理 vs 显式 CoT	在隐空间里多算几步 vs 把中间步骤写成文字 token
不动点（fixed point）	再循环一次输出也不变的状态；一派工作把循环训成朝不动点收敛以保证稳定
PLT / CLP	并行循环 Transformer / 跨循环位置偏移——把上一轮状态右移一位，使不同轮次能并行执行

解读 03 · Training-Free Looped Transformers：给冻结模型套一个循环外壳

项目	内容
论文	Training-Free Looped Transformers
arXiv	2605.23872（2026-05）
作者	Lizhang Chen*、Jonathan Li*、Chen Liang、Ni Lao、Qiang Liu（德州大学奥斯汀分校）
本仓库	原文 `papers/original/2605.23872.pdf` · 中译 `papers/zh/2605.23872.zh.pdf`

读前速览

> **问题** · 已经训练好、发布出来的开源模型，能不能不改一个参数、不加训练，仅靠推理时” 循环” 变强?
> **做法** · 把中间 4 层看作一段 ODE：不要把这段” 走 K 遍”，而是把这一段拆成 K 小步走（Runge–Kutta 风格，加锚点 β）；MoE 用逐层模式固定路由。
> **结果** · 7 个模型家族、45 个（模型，基准）格子 87% 非负；Qwen3-4B MMLU-Pro +2.64、GPQA +2.01，Qwen1.5-MoE ARC-C +2.30。
> **局限** · 收益上限约 2–3 点，集中在知识型多选题；3B 以下蒸馏模型可能掉分；多 15–30% 推理延迟。
> **大白话** · 同一段路，把一大步拆成几小步走得更准；但别顺着惯性多走到别处——那正是朴素循环掉点的原因。



图：朴素循环把状态推到网络没见过的 *t=3*，子步法用三小步更准地走到同一个 *t=1*。

一句话

不改一个权重、不加一个参数，在推理时把已发布模型中间连续 4 层当作一个 ODE 的向量场，用 K 步阻尼欧拉（Runge–Kutta 风格）重新积分到同一个终点；在 Qwen3、Llama-3.2、Qwen1.5-MoE、Moonlight、DeepSeek-V2-Lite 七个模型家族的 45 个（模型，基准）格子上，87% 非负，MMLU-Pro、GPQA 这类知识型多选题提升约 2 个百分点。

1. 要解决的问题

此前所有循环 Transformer 都必须” 为循环而训练”：参数在循环结构下绑定和优化，循环与权重不可分。这意味着几乎所有公开发布的检查点（它们都是多阶段流水线的终点：继续预训练、SFT、多轮 RLHF/DPO）都无法从循环中受益。

论文的问题是：**能否对一个冻结的、开箱即用的检查点直接在推理时循环，不做微调、不做继续训练、不加辅助参数、不改架构?**

三条已有证据指向” 可以”：Men et al. (2025) 删掉整段中间层损失很小；Lad et al. (2024) 中间层可以跳过、交换、重复而不崩；Belrose et al. (2023) 的 tuned lens 显示中间层的 logits 已经编码了大部分最终预测。主流解读是” 中间层冗余，所以可以删”，作者补上另一面：**同样的冗余让中间层可以被安全地重复应用**。

2. 核心方法

2.1 循环外壳

设模型 $f = L_{N-1} \circ \cdots \circ L_0$，选一个连续窗口 $[a, b]$，令 $g = L_b \circ \cdots \circ L_a$。包装后的模型为

$$\hat{f}(x) = \underbrace{(L_{N-1} \circ \cdots \circ L_{b+1})}_{\text{post-loop}} \circ g^{(K)} \circ \underbrace{(L_{a-1} \circ \cdots \circ L_0)}_{\text{pre-loop}}(x)$$

额外代价是 $(b-a+1)(K-1)$ 次层前向。$g^{(K)}$ 有两种实现方式：

- **block-mode**：$(L_b \circ \cdots \circ L_a)^K$，整个窗口作为整体重复 K 次。
- **layer-mode**：$L_b^K \circ \cdots \circ L_a^K$，每层各自重复 K 次再往下传。

稠密模型两种模式差别不大。**MoE 模型必须用 layer-mode**：block-mode 下每次迭代门控网络看到略有不同的隐状态，路由到不同专家，路由噪声累积后盖过了表示层面的精化（routing thrash）。layer-mode 只做一次门控决策、同一专家组合应用 K 次。

2.2 关键洞察：一层 = 一步前向欧拉

pre-norm Transformer 层 $L(x) = x + F(x)$，其中 F 是注意力加 MLP 的残差更新。定义窗口残差场 $F_g(x) := g(x) - x$，则 $g(x) = x + 1 \cdot F_g(x)$ 正好是自洽 ODE

$$\dot{x} = F_g(x)$$

上步长 $h = 1$ 的一步前向欧拉。后面的 post-loop 层是在 $x(t{=}1)$ 的这个近似上训练出来的。

朴素循环为什么掉点：$x \leftarrow g(x)$ 做 K 次，等价于步长 1 走 K 步，到达的是 $x(t{=}K)$，而 post-loop 层从未见过 $t = K$ 处的轨迹。作者用一个隐维度为 2 的小网络可视化（Figure 2）：朴素 K 循环的终点随 K 增大一路漂出训练时的低损失谷；$K = 2$ 时 MSE 从 0.015 涨到 2.88，$K = 8$ 时涨到 335。

正确的目标不是积分到 $t = K$，而是把 $[0, 1]$ 这段积分做得更精细。用 K 步步长 $h = 1/K$ 的欧拉：

$$x_{k+1} = \left(1 - \tfrac{1}{K}\right) x_k + \tfrac{1}{K}\, g(x_k)$$

它以 $O(1/K)$ 阶收敛到真实的 $x(t{=}1)$，严格优于网络原本隐含的单步欧拉。小网络实验里 $K = 8$ 的子步法 MSE 为 1.04，而朴素循环是 335（约 320 倍差距）。

2.3 一个特别有效的 Runge–Kutta 表

把阻尼欧拉推广到 s 级显式 RK，$x_1 = x_0 + h \sum_i b_i k_i$。作者选了一组特定 Butcher 系数：$h = 1$，$s = K$，$a_{ij} = \frac{1}{K}\,\mathbb{I}(j < i)$，输出权重

$$b_1 = \beta + \frac{1-\beta}{K}, \qquad b_i = \frac{1-\beta}{K}\ (i \geq 2)$$

此时 RK 输出恰好等于

$$x_1 = \beta\, g(x_0) + (1-\beta)\, F^K(x_0)$$

即原始单步输出与 K 步阻尼欧拉终点的插值（Algorithm 3）。$\beta = 0$ 退化为纯子步欧拉，$\beta = 1$ 保持原模型不变；中间的 β 把轨迹拉向训练时的一步终点。这个" 锚点权重" 是整套方法的安全阀。

作者还比较了 7 种迭代策略（Table 1）：朴素循环、前向欧拉、中点法/Heun（RK2）、RK4、heavy-ball、Anderson 加速、Aitken Δ^2、Uniform Loop。结论：**没有任何不动点加速器稳定优于 K 级 RK**，说明预训练 Transformer 的中段" 在任何有用的意义上都不是压缩映射"——它不是在逼近一个不动点，把它当 ODE 的一段积分比当不动点迭代更贴切。

3. 关键实验与数字

统一配方，**逐格子零调参**：3 级 RK、中间 4 层窗口、稠密用 block-mode、MoE 用 layer-mode；lm-eval-harness 0.4.11，bf16。约 2 万 H100 小时。

3.1 主结果（Table 3）

模型	基准	基线	循环后	Δ pp
Qwen3-4B-Instruct	MMLU-Pro 5-shot	0.5714	0.5979	**+2.64**
	GPQA-Main 0-shot	0.3371	0.3571	**+2.01**
	CommonsenseQA 7-shot	0.7887	0.7993	+1.06
Llama-3.2-3B-Instruct	GPQA-Main	0.2991	0.3103	+1.12
	MMLU-Pro	0.3164	0.3236	+0.71
Llama-3.2-1B-Instruct	GPQA-Main	0.2790	0.2969	+1.79
Qwen1.5-MoE-A2.7B	ARC-Challenge 25-shot	0.4829	0.5060	**+2.30**
	CommonsenseQA	0.7961	0.8133	+1.72
	OpenBookQA	0.3160	0.3320	+1.60
Moonlight-16B-A3B	MMLU 0-shot	0.6786	0.6860	+0.74
	OpenBookQA	0.3160	0.3280	+1.20
DeepSeek-V2-Lite-Chat	ARC-Challenge	0.5794	0.5879	+0.85

最大收益（+2.0 到 +2.6）出现在最难的知识型多选题（MMLU-Pro、GPQA、ARC-C）和最强的 MHA 骨干上；MLA+MoE 模型（DeepSeek-V2-Lite、Moonlight）方向一致但幅度小 3–4 倍。

3.2 窗口大小：$n = 4$ 是甜点，$n \geq 6$ 是悬崖（Table 2a）

Qwen3-1.7B-Base，前向欧拉 $K = 2$，16 任务平均：

窗口层数 n	1	2	3	**4**	6	12	28（全部）
Δ	+0.18	+0.06	+0.21	**+0.55**	−0.82	−0.63	−27.73

$n = 4$ 在 Qwen3-1.7B/4B、Llama-3.2-3B 上都是赢家（0.6B 上 $n = 3$ 略好）。循环整个模型直接崩掉 27.7 点，这与" 循环整模型" 范式在训练过的模型上有效形成鲜明对照——**没训练过的模型只能在中段小窗口里循环**。

3.3 深度分数规则（Section 3.2）

八个架构上，最优窗口中心都落在一个窄的相对深度带：大于 1.7B 的模型在 0.43–0.71（众数约 0.50），小于 1B 的模型前移到 0.25–0.56。假说：注意力头的特化集中在后段层，循环必须避开；小模型和蒸馏模型的" 后段" 比例更大，安全窗口因此更早。

3.4 泄漏检验：Qwen3-30B-A3B（Table 4）

配置 $[22, 24]$ 完全由深度分数规则事先确定、任何格子打分之前就已锁定。9 个基准 8 个提升：CSQA +1.14、MMLU-flex +0.79、SuperGPQA +0.70、TruthfulQA +0.49；LAMBADA 困惑度 4.12 → 4.11。全部为正但幅度小，说明规则能迁移、也说明大 MoE 上收益有限。

3.5 全景统计（Section 3.5）

45 个格子：27 个正（Δ > +0.3，60%）、12 个中性（27%）、6 个负。负值集中在 **3B 以下的蒸馏模型做知识型多选题**（如 Llama-3.2-1B 的 MMLU）。即使在这个区域，GPQA 在 Llama-3.2-1B 上换到极早窗口 $[4, 7]$ 也能转正（+1.79）——失败边界依赖任务而非绝对。

3.6 与其他循环方法的对比（Section 3.7、Figure 4–5）

- Giannou et al. (2023) 式的朴素循环在每一个格子上崩溃；$K = 6$ 时 16 任务均值掉到 37.89%（−17.71 pp）。
- Uniform Loop（Lys et al. 2026）在 $K \geq 6$ 后失效。
- 本文方法在 $K \in \{1, \ldots, 24\}$ 上保持稳定。

4. 为什么重要

1. **降低了循环深度的门槛。**此前循环是" 从头训练、大厂专利"；本文让任何人在已有开源模型上免费拿到 1–2.5 个百分点。
2. **给出了一个可检验的理论解释。**"层 = ODE 的欧拉步" 不是新观点（Neural ODE、DEQ 早有），新的是把它变成推理时可用的处方：不改变积分终点、只加细步长，并用 β 锚定。这个视角同时解释了为什么朴素循环失败、为什么 MoE 要 layer-mode、为什么不动点加速器无效。
3. **layer-mode 对 MoE 的必要性**与 Loopie（2607.16051）独立得出的"layer-loop 在 MoE 上扩展性更好" 互相印证；两篇论文一篇不训练、一篇从头训练，指向同一个结论。

5. 局限

- 收益上限约 2–3 个百分点，与从头训练的循环模型（Huginn 上 ARC-E +20 点）不在一个量级。它是" 榨干已有权重"，不是" 新的扩展轴"。
- 收益集中在知识型多选题；论文没有报告生成式任务（GSM8K、代码）上的系统性提升。
- 额外推理成本约 $4 \times (K-1)$ 层前向（$K = 3$ 时相当于多 8 层），在 28–48 层模型上是 15–30% 的延迟。
- 小蒸馏模型上会掉分，需要按模型家族调窗口位置；"零调参" 是相对于逐格子而言，配方本身仍是在这些模型上摸出来的。

6. 与本方向其他工作的关系

- 理论根基：DEQ（Bai et al. 2019, 2020）把权重绑定网络视为不动点求解；本文反其道，证明预训练网络的中段更像 ODE 流而非压缩映射。
- 免训练深度操控的近亲：层跳过（Men 2025、Lad 2024）、Program-of-Layers（Li et al. 2026，ICML）——后者学一个小网络决定对每个输入跳过或重复哪些层，可视为本文的可学习版本。
- 改造式循环：Relaxed Recursive Transformers（Bae 2025，逐层 LoRA）、Retrofitted Recurrence（McLeish 2025，课程式继续训练）——这些需要训练但收益更大，与本文构成" 零成本—低成本—从头训" 的连续谱。

7. 对从业者的启示

- 想在已有模型上试循环：中间 4 层、3 级 RK、β 取中间值、稠密 block-mode、MoE layer-mode。先在知识型多选题上验证，这里最容易看到信号。
- 不要朴素地重复层。任何" 把某段层跑两遍" 的实验，都应该用阻尼更新 $x \leftarrow (1-1/K)x + (1/K)g(x)$ 替代 $x \leftarrow g(x)$。
- 对 MoE 模型做任何形式的层重复，先固定路由。
- 这个方法对" 模型已经很强、卡在最后几个点" 的场景最有价值；对小模型或需要生成式推理的场景，去看从头训练的方案。

附：关键数字速查

- 配方：中间 4 层窗口、$K = 3$ 级 RK、稠密 block / MoE layer。
- 45 格子：60% 正、27% 中性、87% 非负；~20k H100 小时。
- Qwen3-4B-Instruct MMLU-Pro +2.64、GPQA +2.01；Qwen1.5-MoE ARC-C +2.30。
- 窗口 $n = 4$ +0.55，$n = 6$ −0.82，全模型 −27.73；朴素循环 $K = 6$ −17.71。
- 最优窗口相对深度：>1.7B 为 0.43–0.71，<1B 为 0.25–0.56。

术语小词典

术语	一句话解释
循环深度（recurrent depth）/ 循环 Transformer	同一组参数在深度方向上重复使用；有效深度 = 存储层数 × 循环次数，参数量不变
model-loop / block-loop / layer-loop	整个层栈循环 / 只循环中间一段 / 每层各自重复几次再下传；见 `assets/figures/diagrams/loop_granularity.png`
输入注入（input injection）	每一次循环都把原始输入的嵌入重新喂给循环块，否则迭代不稳定
截断反传（truncated backprop）	只对最后 k 步循环计算梯度，让显存与循环次数无关
自适应停机（ACT / adaptive halting）	让每个 token 自己决定循环几次，难的多算、简单的早停
路径独立（path independence）	从不同的随机初始状态出发，循环最终收敛到相同的行为
KV cache	注意力缓存的键值；循环模型每循环一次都会多一份，是显存瓶颈
MoE（混合专家）	每个 token 只激活一小部分参数（专家）；"20B-A2B" = 总参数 20B、激活 2B

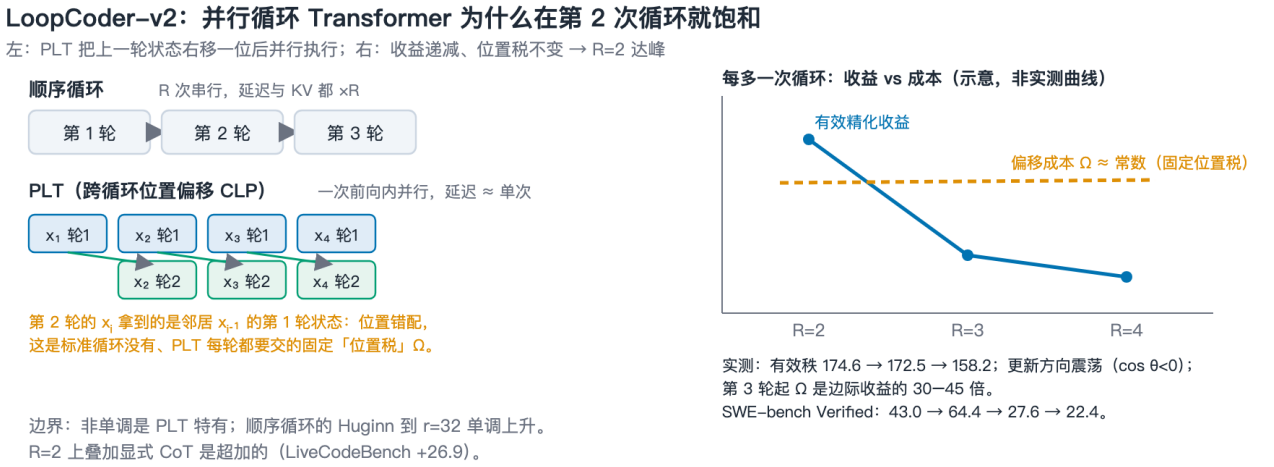
术语	一句话解释
FLOPs 匹配 vs 实测算力匹配	前者按理论计算量对齐，后者按实测训练步时对齐——循环模型省下的激活显存只在后者里体现
潜在推理 vs 显式 CoT	在隐空间里多算几步 vs 把中间步骤写成文字 token
不动点（fixed point）	再循环一次输出也不变的状态；一派工作把循环训成朝不动点收敛以保证稳定
PLT / CLP	并行循环 Transformer / 跨循环位置偏移——把上一轮状态右移一位，使不同轮次能并行执行

解读 04 · LoopCoder-v2：在并行循环 Transformer 里，第二次循环几乎就是全部

项目	内容
论文	LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling
arXiv	2606.18023（2026-06）
作者	Jian Yang、Shawn Guo、Wei Zhang、Tianyu Zheng、Yaxin Du、Haau-Sing Li、Jiajun Wu、Yue Song、Yan Xing、Qingsong Cai、Zelong Huang、Chuan Hao、Ran Tao、Xianglong Liu、Wayne Xin Zhao、Mingjie Tang、Weifeng Lv、Ming Zhou、Bryan Dai（北京航空航天大学 / IQuest Research / 澜舟科技 / 中国人民大学）
模型	Multilingual-Multimodal-NLP/LoopCoder-V2
本仓库	原文 `papers/original/2606.18023.pdf` · 中译 `papers/zh/2606.18023.zh.pdf`

读前速览

问题 · 并行循环 Transformer（PLT）把循环的延迟和显存压平了，那部署时到底该循环几次？为什么？
做法 · 同一配方从头训练 R=1/2/3/4 四个 7B 代码模型（18T token），同一份 SFT，再逐轮诊断隐状态、注意力和输出分布。
结果 · R=2 把 SWE-bench Verified 从 43.0 提到 64.4；R=3 掉到 27.6、R=4 掉到 22.4。有效秩在第 2 轮达峰；位置偏移的固定成本是第 3 轮起边际收益的 30–45 倍。
局限 · 结论是 PLT 特有的（顺序循环没有这笔”税”）；每个 R 单独训练、单种子；没有去掉位置偏移的对照。
大白话 · 多想一遍很值；但这种”并行想法”每多想一遍都要交一笔固定的税，第三遍起就不划算了。



图：左为顺序循环与 *PLT* 并行执行的区别，右为”收益递减、成本不变”的示意。

一句话

用相同配方在 18T token 上从头训练循环次数 $R \in \{1,2,3,4\}$ 的四个 7B 并行循环 Transformer（PLT）代码模型，再做相同的指令微调：$R=2$ 把 SWE-bench Verified 从 43.0 提到 64.4、Multi-SWE 从 14.0 提到 31.0，而 $R=3$ 掉到 27.6、$R=4$ 掉到 22.4。逐循环诊断表明第二次循环承担了几乎全部有效精化，此后每次循环带来的收益迅速萎缩，而 PLT 为了并行引入的跨循环位置偏移（CLP）却是一笔固定的”位置税”，税额比第 3 次以后的边际收益大 30–45 倍。

2. 背景：为什么要用 PLT

标准顺序循环有个部署层面的致命问题：R 次循环意味着 R 次串行通过共享块，延迟乘 R；每层每次循环都缓存 KV，显存 $O(R \cdot L \cdot S \cdot d)$，也乘 R。

Parallel Loop Transformer（PLT，Wu et al. 2025，arXiv 2510.24824）用两个机制把两项成本都压到近似常数（Table 1）：

	顺序循环	PLT
执行	串行	并行、单次前向
延迟	$O(R\,C_{\text{block}})$	$\approx C_{\text{block}}$
KV cache	$O(RLSd)$	$O(LSd)$
循环间输入	$\mathbf{h}^{(r-1)}$	$\text{Embed}(x) + \text{shift}(\mathbf{h}^{(r-1)})$

1. **共享 KV + 门控滑窗注意力（G-SWA）**。第一次循环的 KV $K_{\text{share}}, V_{\text{share}}$ 冻结并共享给后续所有循环；后续循环的每个注意力层输出是全局分支（对冻结 KV 做全上下文注意力）与局部分支（对当前循环 KV 做宽度 $w = 64$ 的滑窗注意力）的逐头门控混合：$\tilde{y}^{(r)} = g \odot y^{(r)}_{\text{global}} + (1-g) \odot y^{(r)}_{\text{local}}$，$g = \sigma(f_{\text{gate}}(\text{RMSNorm}(\mathbf{h})))$。
2. **跨循环位置偏移（CLP）**。第 $r \geq 2$ 次循环的输入是 $B^{(r)} = \text{Embed}(x) + \text{shift}(\mathbf{h}^{(r-1)})$，其中 shift 把上一循环的隐状态整体**右移一个位置**。这样 token x_i 的第 r 次循环可以与 x_{i-1} 的第 $r+1$ 次循环在同一次前向里并行算。

代价：token x_i 在第 r 次循环拿到的不是自己上一轮的状态，而是邻居 x_{i-1} 的。这就是论文所称的**位置错配（positional mismatch）**——PLT 特有、标准循环没有的信息约束。

3. 研究问题与方法

PLT 让循环次数变成一个可以自由选择的部署超参，问题随之而来：选几次？穷举训练太贵，也不解释为什么。作者提出”收益—成本”视角：每多一次循环，一边是表示精化的边际收益，一边是 CLP 的固定错配成本。

3.1 宏观实验协议

- 7B 稠密 PLT，G-SWA 窗口 64，第一循环 KV 共享。
- 从头预训练 18T token，文本: 代码 1:1，代码覆盖 100+ 语言；Adam(0.9, 0.95)，$\epsilon = 10^{-15}$，权重衰减 0.1，学习率 4×10^{-4} 余弦衰减，5% 预热；bf16；总计 **100 万 GPU 小时**。
- 训练与推理的循环次数一致（训 $R = r$ 就测 $R = r$）。
- 四个模型用同一份 600 万条 SFT 数据做指令微调，报告最终检查点。
- 基础设施：定制 Megatron-LM，R 次循环展开为 $R \cdot L$ 个调度层但只实例化一份参数；同一层的所有循环实例放在同一流水线阶段，权重共享零通信。

3.2 微观诊断工具（Section 3.2）

四组逐循环指标，只有多组指标一致时才认定机制成立：

- **隐状态动力学**：步长 $\delta^{(r)} = \|\mathbf{h}^{(r)} - \mathbf{h}^{(r-1)}\|$；相邻更新方向夹角余弦 $\cos\theta^{(r)}$（<0 表示方向反转、震荡）；有效秩 erank（归一化奇异值的熵指数，衡量表示多样性）；不动点差 $\Delta^{(r)}_{\text{FP}} = \|\mathbf{h}^{(r)} - f_\theta(\mathbf{h}^{(r)})\|$。
- **注意力演化**：注意力熵、循环间注意力 KL 散度 $D^{(r)}_{\text{KL}}$、头间余弦相似度、G-SWA 门控均值 $\bar{g}^{(r)}$。
- **输出分布偏移**：logit lens 下真值 token 的排名、循环间输出 KL $\Delta p^{(r)}$、输出熵。
- **固有偏移成本** $\Omega^{(r)} = \frac{1}{S}\sum_i \|\mathbf{h}^{(r-1)}_i - \mathbf{h}^{(r-1)}_{i-1}\|$：相邻 token 表示的平均距离。相邻 token 越不像，把邻居状态当自己状态的失真就越大。

4. 关键结果

4.1 主表（Table 2，7B 模型四个变体）

模型	HE+	MultiPL-E	BCB	LCB	SWE-V	Multi-SWE	TB-v1	TB-v2	M2W	BFCL	均值
基线 $R=1$	81.1	69.5	40.1	27.4	43.0	14.0	26.3	11.2	35.3	32.2	38.0
$R=2$	**84.1**	**73.9**	**46.1**	**35.4**	**64.4**	**31.0**	**34.2**	**21.0**	34.5	**40.1**	**46.5**
$R=3$	75.0	69.8	43.3	28.6	27.6	11.0	30.0	12.2	35.1	36.3	36.9
$R=4$	76.8	67.3	40.8	24.5	22.4	9.3	26.3	9.0	**41.4**	39.5	34.3

三个观察：

- **强非单调**。$R=2$ 均值 +8.5；$R=3$ 比基线还低 1.1；$R=4$ 再低。SWE-bench Verified 43.0 → 64.4 → 27.6 → 22.4。

- **一次额外循环让 7B 追上大得多的模型。** 64.4 的 SWE-bench Verified 超过 Kimi-Dev-72B（60.4）、Qwen3-235B-A22B（45.2），接近 Qwen3-Coder-480B（67.0）。留出的 agentic 基准 SWE-bench-CC 上达到 33.4%。
- **收益集中在 agentic 与推理型任务。** SWE、Terminal-Bench、LiveCodeBench 涨幅最大；Mind2Web 反而是 $R = 4$ 最好——单一任务的噪声，作者没有过度解读。

4.2 诊断：第 2 次循环做了什么、第 3 次为什么坏（Section 4.2，Table 3）

PLT4 模型上 500 条样本的逐循环统计：

循环	步长 δ	输出偏移 Δp	有效秩	$\cos\theta$
$r = 2$	**846**	**1.75**	**174.6**	−0.72
$r = 3$	464	1.32	172.5	−0.46
$r = 4$	1014	1.58	158.2	0.04

- **第 2 次循环是主要的有效精化点**：最大的注意力重路由 $D_{\mathrm{KL}}^{(2)}$、最大的输出偏移、有效秩峰值。
- **第 2 次之后收益坍塌**：Δp 和 D_{KL} 骤降；有效秩从峰值下滑——后续循环在收窄表示子空间而非丰富它；注意力头越来越相似（Figure 4）；隐状态更新方向反转（$\cos\theta < 0$），是震荡而非收敛。四循环模型里第 3 次循环几乎是" 死通道"，第 4 次的大步长只是输出读出，不是新的表示精化（其有效秩最低）。
- **门控始终 >0.5**：后续循环主要依赖冻结的第一循环全局 KV，没有构建定性上新的上下文。
- **偏移成本是固定税**：$\Omega^{(r)}$ 在各循环几乎不变，而每循环收益 $\Delta p^{(r)}$ 在第 2 次后坍塌。Figure 3 的" 剪刀图"：每一次额外循环里，偏移成本高出边际收益 **30–45 倍**。

结论：收益递减 + 固定成本 → 净效应在 $R = 2$ 处达峰，之后转负。

4.3 显式 CoT 与隐式循环互补（Table 4）

在最优的 $R = 2$ 上，比较指令模型（只有隐式循环）和思维模型（显式 CoT + 循环）：

$R = 2$	LCB	CRUX	MultiPL-E	FullStackBench	BCB-Hard
Instruct（仅循环）	35.4	86.9	73.9	47.2	23.7
Thinking（CoT + 循环）	**62.3**	**93.5**	**77.8**	**49.9**	**26.4**
Δ	+26.9	+6.6	+3.9	+2.7	+2.7

LiveCodeBench +26.9 远超两者单独贡献之和：显式 CoT 在非循环模型上不提升这些任务，循环单独只带来个位数收益。作者的解释是两者作用于不同粒度——CoT 把问题拆成文本步骤，循环精化每一步底层的表示——组合后**超加性**。这是本文最有前瞻性的结论：**隐式深度循环与显式推理是互补的测试时算力轴，不是替代品**。

5. 这个结论的适用边界（重要）

"循环两次就够、三次以上回退" 这句话在传播中经常被去掉前提。论文自己划定的边界很清楚：

1. **它是 PLT 特有的现象**。非单调的机制解释依赖 CLP 偏移带来的固定位置税；标准顺序循环没有这一项。Huginn（顺序循环、输入注入）从 $r = 4$ 到 $r = 32$ 单调上升，Ouro 用 4 步，Fixed-Point Reasoners、STARS 等工作专门研究如何让几十步循环稳定。
2. **训练与推理循环次数绑定**。每个 R 都是单独从头训练的模型，不是一个模型在不同 R 下测试。这里的 R 是训练配方的一部分，不是测试时可调的算力旋钮。
3. **与 Loopie 的 $R = 2$ 结论殊途同归但理由不同**。Loopie 选 2 是因为在固定预训练算力下边际收益衰减最快；LoopCoder-v2 选 2 是因为 CLP 的固定成本。两篇论文出自同一批作者（Bryan Dai、Ran Tao 同时出现在两篇作者名单上），可以视为同一团队从" 算力匹配" 和" 机制诊断" 两个方向给出的一致答案。
4. **有效秩是一个可操作的诊断量**。论文建议：候选循环处有效秩还在上升，就还值得多循环一次；开始下降，之后的循环基本只在交位置税。这个诊断不需要穷举训练。

6. 局限

- 每个变体一次训练、单一种子；1M GPU 小时的预算决定了不可能做多种子重复。$R = 3$ 在部分任务上低于基线的幅度（SWE −15.4）大到不像噪声，但个别基准（M2W）的反向波动提醒读者主表存在方差。
- 只有 PLT 一种架构、一种规模（7B 稠密）。"两次最优" 是否随规模、随 MoE 骨干变化未知（SMELT 2609.01343 在 MoE 上给出的答案同样是" 循环中间一半两次"）。

- 没有把 CLP 去掉做对照（顺序循环的 $R = 3$ 会怎样）。论文的机制解释因此是" 一致证据" 而不是因果实验。
- 论文没有公开 18T 预训练数据配方，只给出代码语言分布。

7. 与本方向其他工作的关系

- 前驱：PLT（Wu et al. 2025）提供架构；LoopCoder-v1 的细节未在本文中展开。
- 同期" 循环次数" 研究：Schwethelm et al. (2026) 的迭代深度扩展律给出 $\varphi = 0.46$——循环 r 次只值 $r^{0.46}$ 份独立参数；Loopie 的 $N\times$ 循环 vs $N\times$ 层对照；Yang et al. (2026, STARS) 观察到性能在某个深度达峰后崩溃并用雅可比谱半径正则化修复。
- 效率路线：MELT（跨循环共享 KV + 门控更新）、Looped Latent Attention（跨循环 KV 低秩压缩）、LT2（线性注意力 + 循环）。PLT 是这条路线里" 把延迟也压平" 的代表。
- CoT 与循环互补的观点在 Xu & Sato (2025) 的形式化比较中有理论呼应：循环擅长确定性并行计算（DAG 求值），CoT 擅长自约化问题的近似推断。

8. 对从业者的启示

- 部署 PLT 类并行循环模型：$R = 2$ 是默认值，一次额外前向换来 SWE 类任务 20 个点的提升，性价比极高。
- 想知道该不该多循环一次，看有效秩曲线，不要穷举训练。
- "循环两次就够" 不是循环深度的普适上限；它是并行化设计的代价。如果延迟预算允许顺序循环，Huginn/Ouro 路线可以把循环用到更深。
- 把隐式循环当成 CoT 的底座而不是替代：先做循环，再叠 thinking 模式，收益超加。

附：关键数字速查

- 7B PLT，18T token，1:1 文本代码，1M GPU 小时；$R \in \{1, 2, 3, 4\}$ 各自从头训练。
- SWE-bench Verified：43.0 → **64.4** → 27.6 → 22.4；Multi-SWE：14.0 → **31.0** → 11.0 → 9.3；均值 38.0 → **46.5** → 36.9 → 34.3。
- PLT4 有效秩：174.6 / 172.5 / 158.2（$r = 2/3/4$）；$\cos\theta$：−0.72 / −0.46 / 0.04。
- 偏移成本 $\Omega \approx$ 常数，是第 3 次起边际收益的 30–45 倍。
- Thinking 相对 Instruct（$R = 2$）：LCB +26.9、CRUX +6.6。

术语小词典

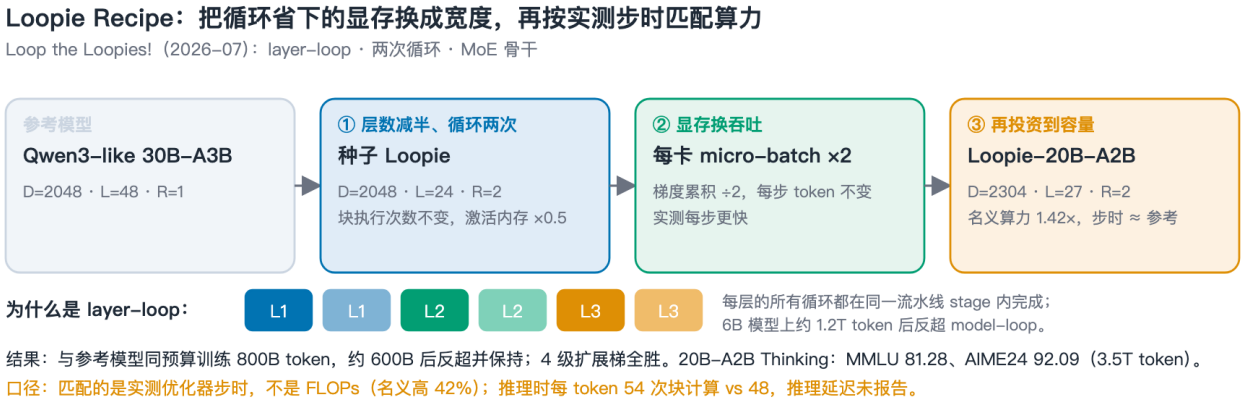
术语	一句话解释
循环深度（recurrent depth）/ 循环 Transformer	同一组参数在深度方向上重复使用；有效深度 = 存储层数 × 循环次数，参数量不变
model-loop / block-loop / layer-loop	整个层栈循环 / 只循环中间一段 / 每层各自重复几次再下传；见 `assets/figures/diagrams/loop_granularity.png`
输入注入（input injection）	每一次循环都把原始输入的嵌入重新喂给循环块，否则迭代不稳定
截断反传（truncated backprop）	只对最后 k 步循环计算梯度，让显存与循环次数无关
自适应停机（ACT / adaptive halting）	让每个 token 自己决定循环几次，难的多算、简单的早停
路径独立（path independence）	从不同的随机初始状态出发，循环最终收敛到相同的行为
KV cache	注意力缓存的键值；循环模型每循环一次都会多一份，是显存瓶颈
MoE（混合专家）	每个 token 只激活一小部分参数（专家）；"20B-A2B" = 总参数 20B、激活 2B
FLOPs 匹配 vs 实测算力匹配	前者按理论计算量对齐，后者按实测训练步时对齐——循环模型省下的激活显存只在后者里体现
潜在推理 vs 显式 CoT	在隐空间里多算几步 vs 把中间步骤写成文字 token
不动点（fixed point）	再循环一次输出也不变的状态；一派工作把循环训成朝不动点收敛以保证稳定
PLT / CLP	并行循环 Transformer / 跨循环位置偏移——把上一轮状态右移一位，使不同轮次能并行执行

解读 05 · Loop the Loopies!：在同等预训练算力下，循环 MoE 第一次赢了普通 MoE

项目	内容
论文	Loop the Loopies!
arXiv	2607.16051（2026-07）
作者	Zitian Gao、Yilong Chen、Yihao Xiao、Xinyu Yang、Ran Tao、Joey Zhou、Bryan Dai（IQuest Research）
模型	Loopie-20B-A2B、Loopie-6B-A0.6B（Base / Thinking）
本仓库	原文 `papers/original/2607.16051.pdf` · 中译 `papers/zh/2607.16051.zh.pdf`

读前速览

> **问题** · 给同样的预训练算力，循环 Transformer 能不能赢过把参数直接做大的普通 MoE?
> **做法** · 逐层循环两次、层数减半 → 激活显存减半 → 每卡 micro-batch 翻倍 → 用省下的时间买更宽的模型；按实测训练步时匹配；SPT + RL 后训练。
> **结果** · 同预算训练 800B token，约 600B 后反超 Qwen3-30B-A3B 复现版；20B-A2B Thinking MMLU 81.28、AIME24 92.09，预训练 token 只有对手的 1/7。
> **局限** · 匹配的是训练步时而非 FLOPs（名义高 42%）；推理成本没报；对照是 800B token 的自复现版。
> **大白话** · 不是靠多循环省参数，而是靠循环省下的显存换来更宽的模型，最后在同样的训练时间里赢。



图：*Loopie Recipe* 三步，以及 *layer-loop* 的执行顺序。

一句话

循环 Transformer 长期被一个质疑压着：给 N 倍预训练算力，把参数扩大 N 倍通常比循环 N 次更划算。Loopie 用三件事回应它：（1）把" 整模型循环" 换成" 逐层循环"（layer-loop），（2）一套按实测训练步时而非理论 FLOPs 匹配算力的配方（Loopie Recipe），（3）只循环两次。结果是 Loopie-20B-A2B 在与 Qwen3-30B-A3B 复现版**相同的每步墙钟时间**下训练 800B token，在约 600B token 后反超并保持领先；配合 3.5T 预训练 + 2T "监督预训练"（SPT）+ 数学/代码 RL，20B-A2B 的 Thinking 版本 MMLU 81.28、AIME24 92.09，用不到 Nemotron 3 Nano（25T token）七分之一的预训练 token 达到相当或更好的成绩。

1. 要解决的问题

论文引言里的核心提问：**在相同的预训练算力预算下，循环 Transformer 能否追平或超过普通 Transformer?**

此前的循环工作（Ouro、Huginn）主要把循环当作参数效率手段：同样参数量下更强。但循环 N 次，训练算力也约乘 N。要证明循环是算力效率手段，就必须和" 把同样算力花在更多独立参数上" 的普通模型比——在这个比较下，以往的循环模型通常输（Csordás 2024、Frey 2026）。加上现代前沿模型几乎都是 MoE（Qwen3、Kimi K2.5、GLM-5、DeepSeek-V4、MiniMax-M2），一个只在稠密模型上有效的循环方案没有通向前沿的路。

2. 核心方法

2.1 Layer-loop：换一种循环顺序

三层、两步循环的例子：

- **model-loop**（Ouro、Huginn 的做法）：Layer1 → Layer2 → Layer3 → Layer1 → Layer2 → Layer3
- **layer-loop**（Loopie）：Layer1 → Layer1 → Layer2 → Layer2 → Layer3 → Layer3

每层先在隐状态上局部迭代，再把结果交给下一层。作者给了三个理由：

1. **实测扩展性更好**。Loopie-6B-A0.6B 上，layer-loop 早期略落后 model-loop，约 1.2T token 后反超并拉开（Figure 2）。
2. **对基础设施友好**。同一层的重复应用在前向和反向图中相邻，参数与梯度的复用距离短，激活检查点和梯度累积更简单；流水线并行下，一层的所有循环都在同一个 stage 内完成，不需要把末尾 stage 的输出送回首个 stage——model-loop 会引入循环依赖、增加通信和流水线气泡。
3. **参数共享模式更自然**。48 层模型做 2 步 model-loop，第 3 层同时要服务有效深度 3 和 51 两个位置；已有分析表明 Transformer 表示沿深度并非均匀分布，前后段与中段差异大。layer-loop 让一层只在相邻的有效深度上复用，共享更" 局部"。

消融（Section 2.7，Figure 5）：同骨干、同数据、同 token、同计算预算，去掉 layer-loop 的 6B-A0.6B 明显更差。改进不能归因于额外计算，而在于循环的调度方式。

2.2 Loopie Recipe：按实测步时匹配算力

预训练主导项 $\mathcal{C}(D, L, R) \propto LD^2R$（宽度 D、存储层数 L、循环数 R），激活内存主导项 $M_{\text{act}} \propto s\,b\,D\,L$——注意激活内存随**存储深度**而非执行深度增长（同一层的所有循环放在同一个检查点单元里）。配方三步：

1. 从非循环参考模型出发（Qwen3-like 30B-A3B：$D_0 = 2048$，$L_0 = 48$，$R_0 = 1$），**层数减半、循环两次**得到种子模型（2048, 24, 2）。块执行次数 $LR = 48$ 不变，主导算力不变，激活内存减半。
2. 把省下的内存换成**每卡 micro-batch 翻倍**（$b_1 = 2b_0$，梯度累积步数减半，每步 token 数不变），实测得到训练效率增益 S_{mb}。
3. 把效率增益**再投资到模型容量**：在种子附近扫 (D, L)，保留能支持 2 倍 micro-batch 的候选，**选实测优化器步时**与参考模型最接近的一个。

配置	D	L	R	归一化算力 $\widehat{C}$	归一化激活内存
Qwen3 30B-A3B（参考）	2048	48	1	1.00×	1.00×
种子 Loopie	2048	24	2	1.00×	0.50×
候选 1	2176	25	2	1.18×	0.55×
候选 2 → Loopie-20B-A2B	**2304**	**27**	**2**	**1.42×**	**0.63×**
候选 3	2432	28	2	1.65×	0.69×

Loopie-20B-A2B 名义上多做 42% 的 Transformer 块计算，但在 Megatron-LM 里（EP=8、MBS=2 对比参考的 EP=8、MBS=1）每步墙钟时间与参考持平。**作者明确说这不是 FLOPs 相等，是实测训练成本相等**——效率增益全部来自 micro-batch 翻倍。

2.3 为什么只循环两次（Section 2.8）

固定存储参数和步数时，多循环几乎总能改善训练曲线，但那不是算力匹配的比较。在固定预算下，多循环意味着更少 token、更小模型、或面对更强的普通基线。作者用 $N\times$ layer-loop 对比 $N\times$ 存储层（250B token，Figure 6）：**循环的边际收益在 $R = 2$ 处最大，之后快速衰减**。

作者同时声明：这不是说更大循环数无效——推理算力便宜、有自适应计算、或研究目的是循环推理本身时，大 R 有其价值；但**前沿预训练的主约束是总算力，此时 $R = 2$ 是迭代精化、吞吐和算力匹配扩展之间最好的折中**。

3. 关键结果

3.1 算力匹配对照（Section 2.5，Figure 3）

Loopie-20B-A2B 与自复现的 Qwen3-30B-A3B 在**同一预训练预算**下各训 800B token。Loopie 早期落后，**约 600B token 后反超并保持优势**（8 个下游基准平均：ARC-C/E、BoolQ、CSQA、HellaSwag、MMLU、OBQA、WinoGrande）。

3.2 扩展梯（Section 2.6，Figure 4）

四级：非循环基线 0.15B / 0.25B / 0.5B / 1B（激活参数）对应算力匹配的 Loopie 0.10B / … / 0.70B，各训 1000× 激活参数量的 token（150B / 250B / 500B / 500B，最后一级受算力限制）。**四级全部 Loopie 领先，且差距不随规模缩小。**

3.3 预训练配方（Section 3）

- 阶段 1：Nemotron-CC-v2-HQ 约 570B 唯一 token，**训 4 个 epoch**（2.28T）——高质量数据多轮训练优于低质量数据单轮。
- 阶段 2：1.26T 高质量退火数据（Nemotron SFT-v1 27.8%、Specialized-v1 21.9%、Code-v2 20.7%、CC-v2-HQ-Synthetic 15.6%、CC-Math-v1 10.0%……）。
- 合计约 3.5T。Qwen3 tokenizer；AdamW，$\epsilon = 10^{-15}$，权重衰减 0.1，学习率 5×10^{-4}（6B）/ 3×10^{-4}（20B），**warmup-stable 无衰减**；batch 1024 × 8192。
- 初始化：嵌入 $\mathcal{N}(0, d^{-1})$，块内参数 SmallInit $\mathcal{N}(0, \frac{2}{5d})$——与 Huginn 采用的 Takase 初始化同源。

3.4 后训练：监督预训练（SPT）+ RL（Section 4）

SPT 是本文第二个方法贡献：用 SFT 的损失掩码（只对目标 token 计损失）配 PT 的优化规模（global batch ≥1024、序列 ≥128K、每 batch ≥1.28 亿 token 位置、总量 2T token、约 10 个 epoch）。

	SPT	SFT	PT
计损失位置	目标 token	目标 token	全部非 padding
预训练指标（MMLU、ARC-C）	↑	↓	↑
推理指标	↑	↑	≈
多 epoch 过拟合	未观察到	2–4 epoch 出现	未观察到
每 batch token 位置	≥128M	128K–1M	8–32M

SPT 同时提升两类指标、10 个 epoch 无 loss 断崖（Figure 9–11）。作者的解释：每次更新聚合上千条序列、上亿 token 位置的监督，抑制了小 batch SFT 里的快速特化和记忆。这一结果直接挑战"SFT 必然导致灾难性遗忘" 的常识。

RL：GSPO（序列级长度归一化重要性比 + 序列级裁剪）加 DAPO 的 clip-higher 与动态采样；先数学后代码，Guru-RL 语料；32K → 64K 响应长度分阶段。

3.5 Thinking 模型成绩（Table 3、4）

Loopie-20B-A2B Thinking（3.5T 预训练 token）对比：

	Qwen3-30B-A3B Thinking (36T)	Nemotron 3 Nano 30B-A3B (25T)	Nemotron Cascade 2 (25T)	GPT-OSS 20B-A2B	**Loopie-20B-A2B (3.5T)**
MMLU	85.83	80.52	81.22	81.64	**81.28**
ARC-Challenge	96.67	91.96	93.86	92.42	93.52
BBH	86.59	68.76	75.86	84.76	82.28
IFEval	85.58	77.05	79.21	70.03	84.72
HumanEval	96.95	86.59	79.88	92.07	89.02
AIME 24 (avg@8)	90.10	85.00	93.33	88.33	92.09
AIME 25	83.75	74.17	91.25	87.50	83.75
AMC	96.73	91.80	93.57	91.05	94.21

用 Nemotron 同源数据的两个 25T 模型作对照：Loopie 在 7 个知识/通用基准里 5 个超过两者，token 用量不到它们的 1/7。

Loopie-6B-A0.6B：MMLU 78.36、**AIME24 80.42**、**AIME25 70.83**（Ouro 2.6B Thinking 为 62.50 / 51.67）、MATH-500 93.80。0.6B 激活参数拿到 80 分的 AIME24 是这张表里最惹眼的数字。

4. 为什么重要

1. **第一次在算力匹配下赢。**这是循环深度从" 学术玩具" 到" 可规模化预训练范式" 的分界。此前的比较不是参数匹配（对循环有利）就是 FLOPs 匹配（对循环不利）；Loopie 用" 实测墙钟时间匹配" 给出第三种、也是对工业界最有意义的标准。
2. **把循环和 MoE 结合。** Huginn 论文第 9 节预测" 循环擅长推理、MoE 擅长存储、未来架构会同时包含两者"，Loopie 是这个预测的第一个大规模实证。
3. **Layer-loop 有独立于本文的旁证。** Training-Free Looped Transformers 在不训练的前提下发现 MoE 必须 layer-mode；SMELT（2609.01343）在 FLOPs/参数/KV 三重匹配下也得出" 中间一半层循环两次" 的配方。三篇论文、三种实验范式、同一个结论。
4. **SPT 可能是独立于循环的贡献。**它没有理由只对循环模型有效；如果在稠密模型上复现，会改变后训练的默认做法。

5. 局限与需要小心的地方

- **"算力匹配" 是实测步时匹配，Loopie 的名义 FLOPs 高 42%。**这个口径对训练成本公平，对推理成本不公平：推理时 Loopie-20B-A2B 每 token 做 27×2=54 次块计算，参考模型 48 次，且循环 KV cache 更大。论文没有报告推理延迟对比。
- 与 Qwen3-30B-A3B 的对照是自复现版，训 800B token（官方 36T）；Table 3 里与官方 Qwen3 Thinking 的差距（MMLU 81 vs 86、BBH 82 vs 87）是真实的。
- 扩展梯最大只到 1B 激活参数；20B-A2B 的算力匹配对照只有一个。
- $N\times$ 循环 vs $N\times$ 层的 Figure 6 中，2× 层基线的实际算力" 显著高于" 2× 循环模型（作者自述），这张图不能读成"2 倍循环被 2 倍层压制"。
- 不公开预训练代码；数据全部来自 Nemotron 公开集，可复现性尚可。

6. 与本方向其他工作的关系

- 前驱：UT（layer-tying）、ALBERT、MoEUT（MoE + UT 的第一次结合，但只在小规模上与标准 Transformer 持平）、Relaxed Recursive Transformers。
- 同团队：LoopCoder-v2（PLT，$R=2$）。两篇论文分别从算力匹配与机制诊断给出 $R=2$。
- 后继 / 同期：SMELT（2609.01343）为算力匹配 MoE 循环拟合 Chinchilla 式扩展律，节省 6.8–18.0% 训练 FLOPs，并发现第二次访问会削弱注意力沉降（attention sink）；Schwethelm et al. 的 $\varphi=0.46$ 解释了为什么" 纯参数换循环" 在 FLOPs 匹配下会输——Loopie 的做法是不做这种交换，而是用内存红利买宽度。
- 与 Hidden Decoding（2607.08186，微信 WeLM 617B）形成对照：后者认为循环与流水线并行不兼容，转而在序列长度维度上加算力；Loopie 的 layer-loop 恰恰是为了解决流水线并行兼容性。这是当前" 怎样给每个 token 加算力" 的两条竞争路线。

7. 对从业者的启示

- 做循环模型的算力对照实验时，按实测步时匹配而不是按 FLOPs：激活内存的节省是循环架构真实存在的红利，应当计入。
- MoE 骨干上默认 layer-loop；从流水线并行的角度这也是更容易实现的选择。
- 在前沿预训练预算下 $R=2$；更大的 R 留给推理时自适应计算。
- SPT 值得在任何模型上试：把 SFT 数据当预训练数据用（大 batch、长序列、多 epoch），同时看预训练指标是否不再下降。
- 循环模型的推理成本仍是未解问题，部署前要实测延迟和 KV cache。

附：关键数字速查

- Loopie-20B-A2B：$D=2304$，$L=27$，$R=2$，名义算力 1.42×，激活内存 0.63×，步时与 Qwen3-30B-A3B 持平（MBS 2 vs 1）。
- 算力匹配对照：800B token，约 600B 后反超。扩展梯 4 级全胜。
- 预训练 3.5T（2.28T × 4 epoch HQ + 1.26T 退火）；SPT 2T、10 epoch；RL GSPO+DAPO，32K→64K。
- 20B-A2B Thinking：MMLU 81.28、ARC-C 93.52、BBH 82.28、IFEval 84.72、AIME24 92.09、AMC 94.21。
- 6B-A0.6B Thinking：AIME24 80.42、AIME25 70.83、MATH-500 93.80。

术语小词典

术语	一句话解释
循环深度（recurrent depth）/ 循环 Transformer	同一组参数在深度方向上重复使用；有效深度 = 存储层数 × 循环次数，参数量不变
model-loop / block-loop / layer-loop	整个层栈循环 / 只循环中间一段 / 每层各自重复几次再下传；见 `assets/figures/diagrams/loop_granularity.png`
输入注入（input injection）	每一次循环都把原始输入的嵌入重新喂给循环块，否则迭代不稳定
截断反传（truncated backprop）	只对最后 k 步循环计算梯度，让显存与循环次数无关
自适应停机（ACT / adaptive halting）	让每个 token 自己决定循环几次，难的多算、简单的早停
路径独立（path independence）	从不同的随机初始状态出发，循环最终收敛到相同的行为
KV cache	注意力缓存的键值；循环模型每循环一次都会多一份，是显存瓶颈
MoE（混合专家）	每个 token 只激活一小部分参数（专家）；“20B-A2B” = 总参数 20B、激活 2B
FLOPs 匹配 vs 实测算力匹配	前者按理论计算量对齐，后者按实测训练步时对齐——循环模型省下的激活显存只在后者里体现
潜在推理 vs 显式 CoT	在隐空间里多算几步 vs 把中间步骤写成文字 token
不动点（fixed point）	再循环一次输出也不变的状态；一派工作把循环训成朝不动点收敛以保证稳定
PLT / CLP	并行循环 Transformer / 跨循环位置偏移——把上一轮状态右移一位，使不同轮次能并行执行